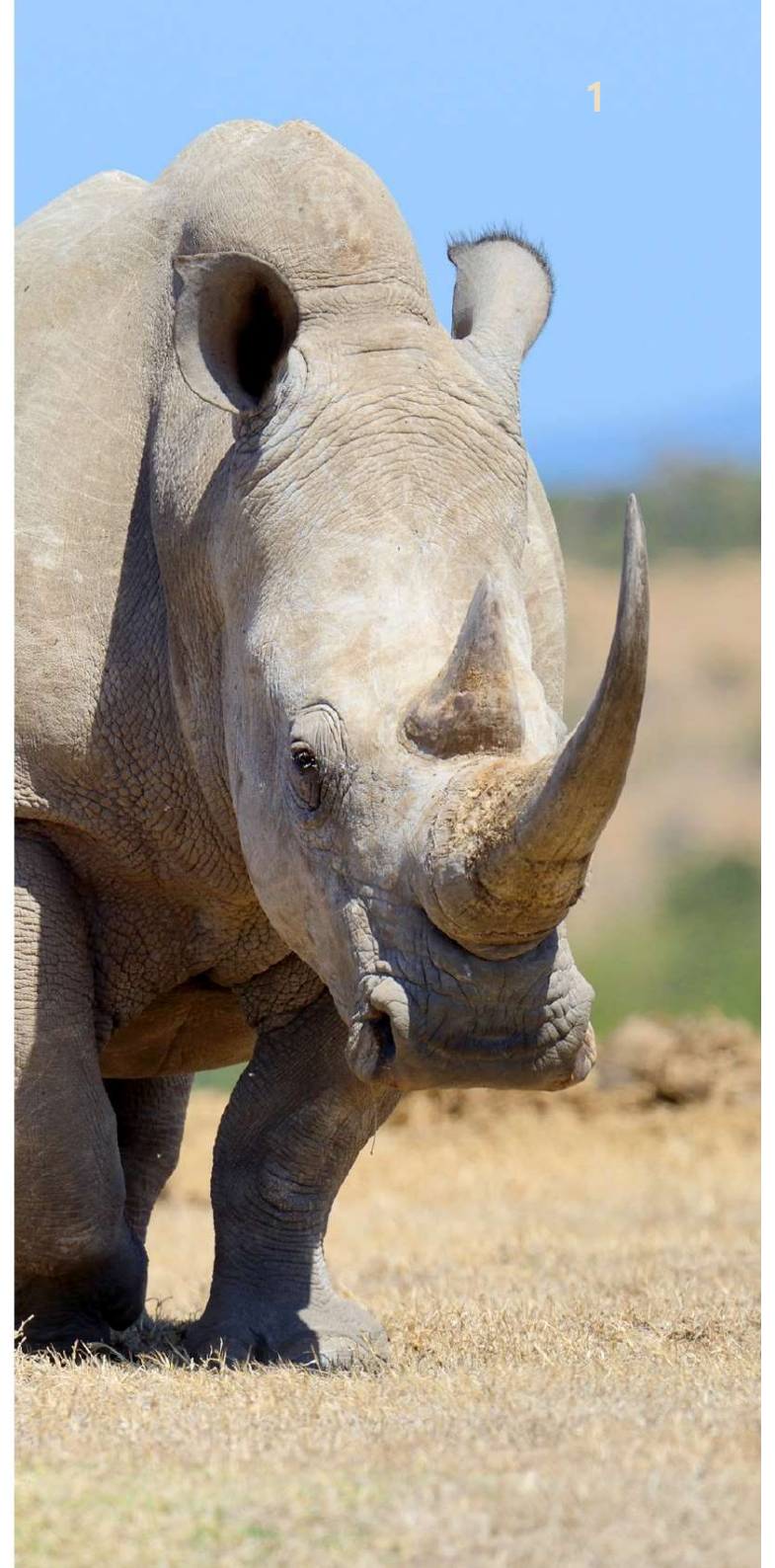


Chapter

04

스레드와 멀티스레딩

1. 프로세스의 문제점
2. 스레드 개념
3. 스레드 주소 공간과 컨텍스트
4. 커널 레벨 스레드와 사용자 레벨 스레드
5. 멀티스레드 구현
6. 멀티스레딩에 관한 이슈



강의 목표

1. 프로세스의 문제점을 인식하고 스레드의 필요성과 개념을 이해한다.
2. 오늘날 운영체제의 실행 단위가 스레드임을 알고 오늘날 운영체제는 스레드 단위로 스케줄함을 안다.
3. 스레드와 프로세스의 차이, 이들의 관계에 대해 이해한다.
4. 멀티스레드 샘플 코드를 통해 멀티스레드 응용프로그램의 작성 방법을 간단히 알고, 스레드의 생성 및 실행 과정을 이해한다.
5. 운영체제가 스레드를 관리하는 방법을 구체적으로 이해한다.
 - 스레드 주소 공간, 컨텍스트, 스레드 제어 블록, 스레드 컨텍스트 스위칭 등
6. 커널 레벨 스레드와 사용자 레벨 스레드의 차이점을 이해한다.
7. 멀티스레딩이 구현되는 3가지 방법에 대해 이해한다.

3

1. 프로세스의 문제점

프로세스의 문제점

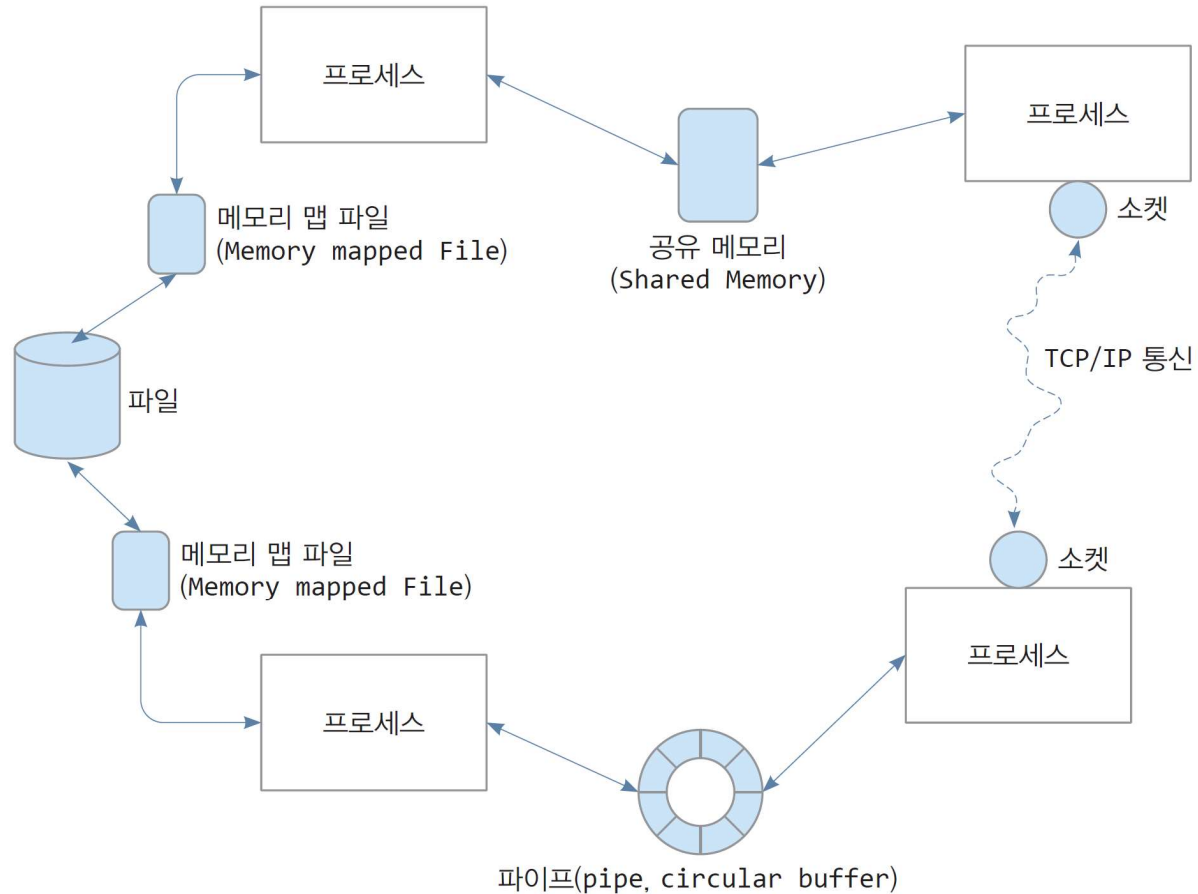
4

- 다중 프로세스를 이용한 멀티태스킹
 - ▣ 응용프로그램에서 여러 프로세스를 생성하여 동시에 여러 작업 실행
 - ▣ 운영체제는 스케줄링을 통해 여러 프로세스(작업)를 번갈아 실행

- 프로세스를 실행 단위로 하는 멀티태스킹의 문제점
 1. 프로세스 생성의 큰 오버헤드
 - 프로세스를 위한 메모리 할당, 부모프로세스로부터 복사
 - PCB 생성, 매핑 테이블(페이지 테이블) 생성 등
 2. 프로세스 컨텍스트 스위칭의 큰 오버헤드
 - CPU 레지스터들을 컨텍스트로 PCB에 저장, 새 프로세스 컨텍스트를 PCB에서 CPU로 옮기는 시간
 - CPU가 참고할 매핑 테이블(페이지 테이블)의 교체 시간
 - CPU 캐시에 새 프로세스의 코드와 데이터가 채워지는데 걸리는 시간 등
 3. 프로세스 사이 통신의 어려움
 - 프로세스가 다른 프로세스의 메모리에 접근 불가
 - 프로세스 사이의 통신을 위한 제 3의 방법 필요
 - 커널 메모리나 커널에 의해 마련된 메모리 공간을 이용하여 데이터 송수신
 - 신호, 소켓, 메시지 큐, 세마포, 공유메모리, 메모리맵 파일 등
 - 이 방법들은 코딩이 어렵고, 실행 속도 느리고, 운영체제 호환성 부족

프로세스 사이의 통신 기법들

5



프로세스 사이의 통신 기법 중 공유 메모리, 신호, 파이프에 관한 자세한 설명은
부록 A(홈페이지에서 PDF 다운로드 가능) 참고

6

2. 스레드 개념

스레드 출현 목적

7

- 프로세스를 실행 단위로 하는 멀티태스킹의 문제점
 - ▣ 커널에 많은 시간, 공간 부담 -> 시스템 전체 속도 저하

- 효율적인 새로운 실행 단위 필요 : 스레드 출현
 - 1) 프로세스보다 크기가 작아,
 - 2) 프로세스보다 생성 및 소멸이 빠르고,
 - 3) 컨텍스트 스위칭이 빠르며,
 - 4) 통신이 쉬운, 실행 단위 필요

스레드 개념(1)

8

□ 스레드는 실행 단위이며 스케줄링 단위

- 스레드는 응용프로그램 개발자에게는 작업을 만드는 단위
 - 하나의 응용프로그램에 동시에 실행할 여러 작업(스레드) 작성 가능
 - 작업은 독립적으로 실행되는 함수로 작성
- 스레드는 운영체제에게 실행 단위이고, 스케줄링 단위
- 스레드는 코드, 데이터, 힙, 스택을 가진 실체
- 스레드마다 스레드 정보를 저장하는 구조체 TCB(Thread Control Block) 있음

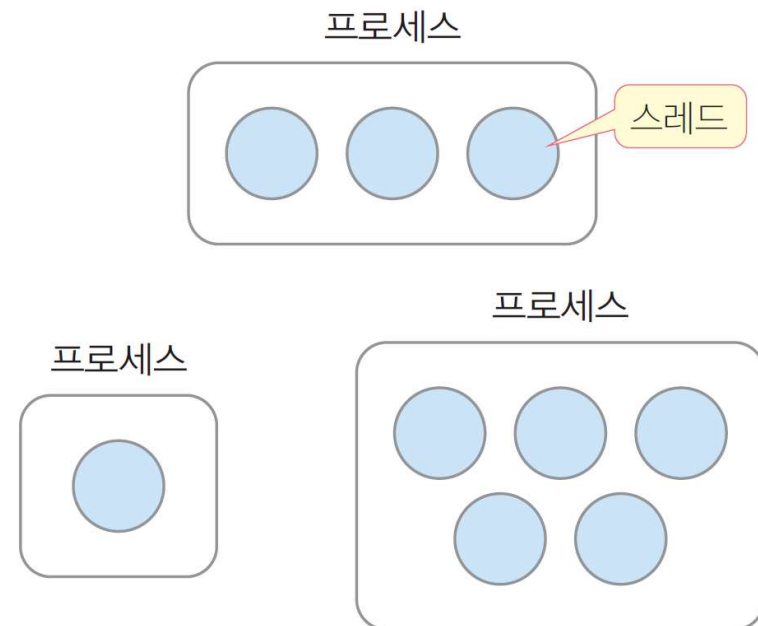
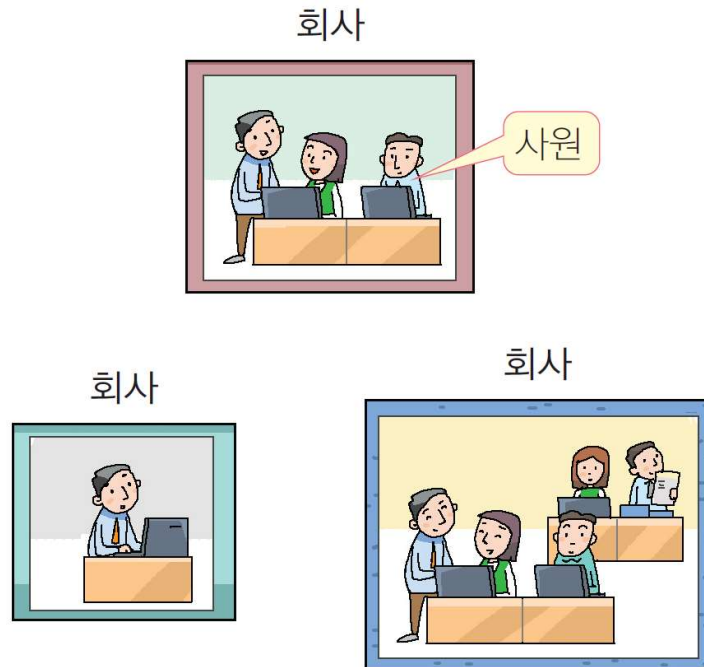
□ 프로세스는 스레드들의 컨테이너

- 프로세스 개념이 스레드들의 컨테이너 역할로 수정됨
- 프로세스는 반드시 1개 이상의 스레드로 구성
 - 프로세스가 생성될 때 운영체제에 의해 자동으로 1개의 스레드 생성 : 메인 스레드(main 스레드)라고 부름
- PCB와 TCB의 관계(그림 참고)

프로세스는 스레드들의 컨테이너

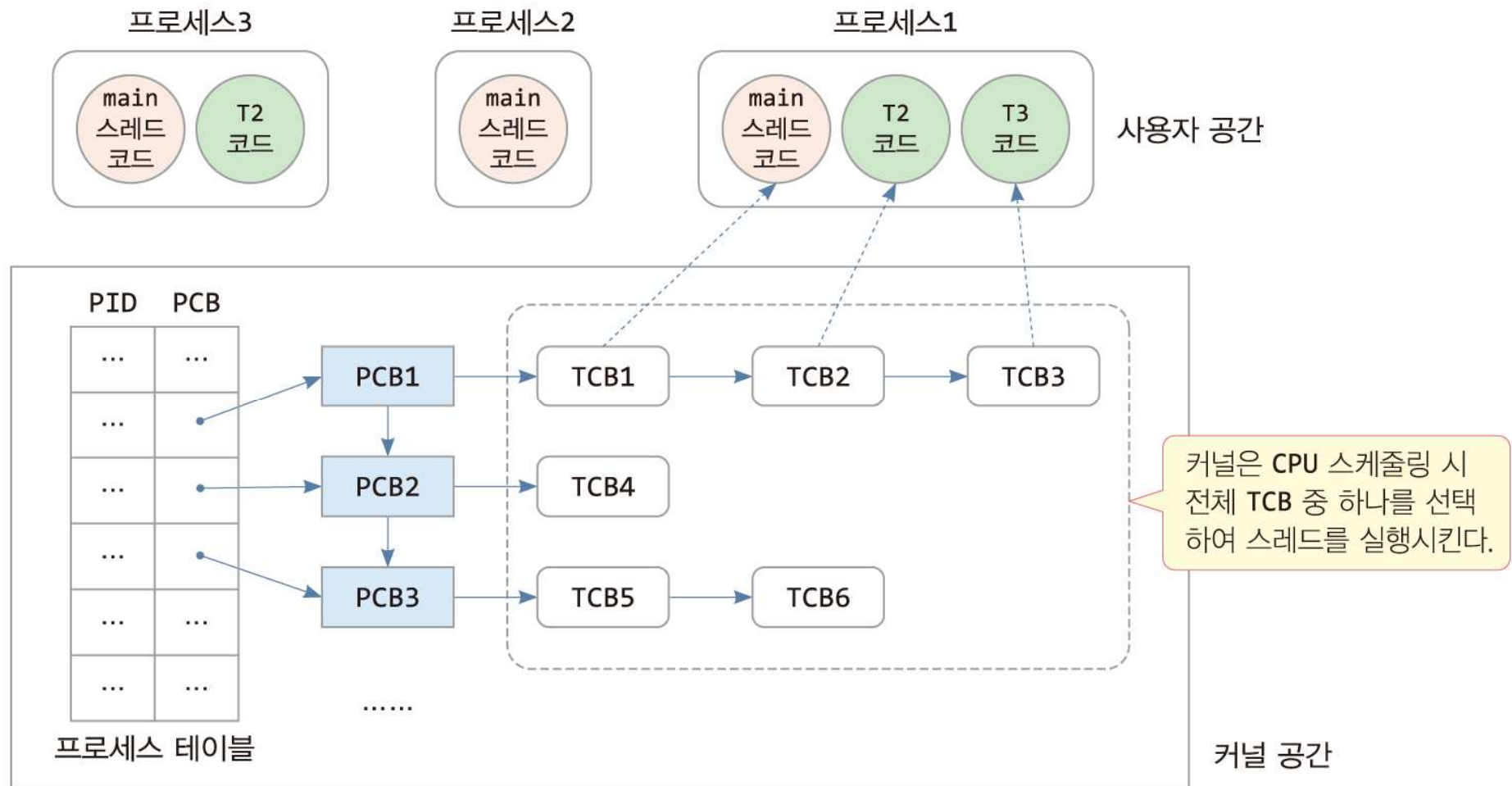
9

- 프로세스는 회사, 스레드는 직원에 비유
 - 직원은 회사의 목적을 위해 일을 하는 단위
 - 스레드는 프로세스(응용프로그램)의 목적을 위해 동시에 실행될 작업(코드) 단위
 - 직원이 3명이면 동시에 일을 하는 사람이 3명인 것처럼, 프로세스에 3개의 스레드가 있으면 3개의 작업이 동시에 실행



프로세스와 스레드 관리 구성

10

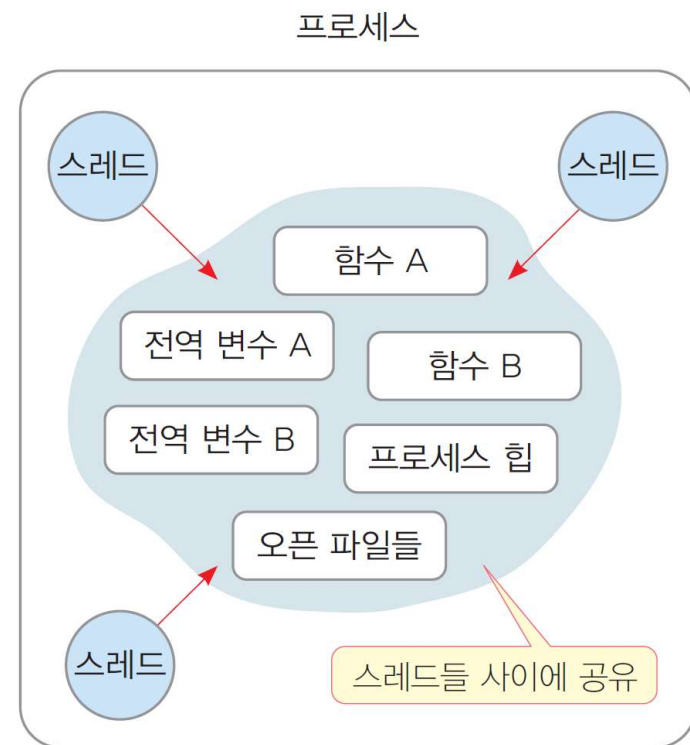
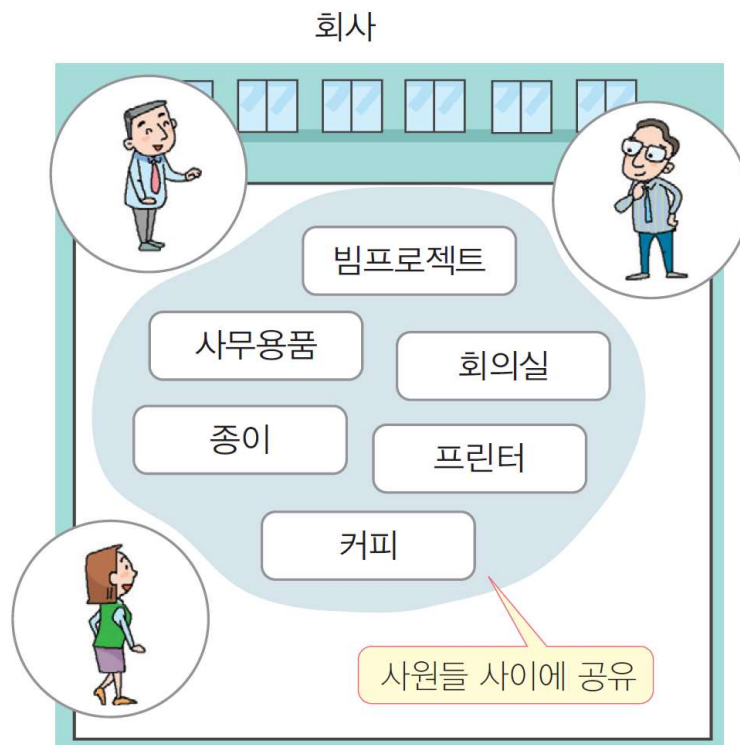


* 스레드마다 TCB가 만들어지고 서로 연결된다. 프로세스에 속한 스레드들을 관리하기 위해 PCB는 TCB와 연결된다.

스레드 개념(2)

11

- 프로세스는 스레드들의 공유 공간(환경) 제공
 - ▣ 모든 스레드는 프로세스의 코드, 데이터, 힙을 공유하며, 프로세스의 스택 공간을 나누어 사용
 - ▣ 공유되는 공간을 이용하면 스레드 사이의 통신 용이



스레드 개념(3)

12

□ 스레드가 실행할 작업은 함수로 작성

- 응용프로그램 개발자는 스레드가 실행할 작업을 함수로 작성
 - 함수를 실행할 스레드 생성을 운영체제에게 요청할 때 스레드 생성
 - 운영체제는 TCB 생성, 함수의 주소를 스레드 실행 시작 주소로 TCB에 등록.
 - 스레드 생성은 곧 TCB 생성
- 운영체제는 TCB 리스트로 전체 스레드 관리
 - 스레드 스케줄 : TCB 중에서 하나 선택, 스레드 단위로 스케줄
 - TCB에 기록된 스레드의 시작 주소를 CPU에 적재하면 실행 시작됨

□ 스레드의 생명과 프로세스의 생명

- 스레드로 만든 함수가 종료하면 스레드 종료
- 스레드가 종료하면 TCB 등 스레드 관련 정보 모두 제거
- 프로세스에 속한 모든 스레드가 종료될 때, 프로세스 종료

스레드 만들기 맛보기 – pthread 라이브러리 이용

13

□ 개요

- ▣ 2개의 스레드로 구성된 멀티스레드 C 응용프로그램 작성
 - 메인 스레드와 calcThread 작성

□ 구성

- ▣ main() 함수
 - main 스레드 코드
 - calcThread 스레드를 생성하여 1에서 100까지 합을 구하게 시키고,
 - calcThread 스레드의 종료를 기다린 후, 합(sum 변수) 출력
- ▣ calcThread() 함수
 - 스레드 코드
 - 정수를 매개변수(param)로 받아 1에서 param까지 합을 구하여 전역 변수 sum에 저장
- ▣ 전역 변수 sum
 - calcThread와 main 스레드 모두 접근

프로그램 코드

14

makethread.c

```
#include <pthread.h> // pthread 라이브러리를 사용하기 위해 필요한 헤더 파일
#include <stdio.h>
#include <stdlib.h>

void* calcThread(void *param); // 스레드로 작동할 코드(함수)
int sum = 0; // main 스레드와 calcThread가 공유하는 전역 변수

int main() {
    pthread_t tid; // 스레드의 id를 저장할 정수형 변수
    pthread_attr_t attr; // 스레드 정보를 담을 구조체

    pthread_attr_init(&attr); // 디폴트 값으로 attr 초기화
    pthread_create(&tid, &attr, calcThread, "100"); // calcThread 스레드 생성
    // 스레드가 생성된 수 커널에 의해 언젠가 스케줄되어 실행

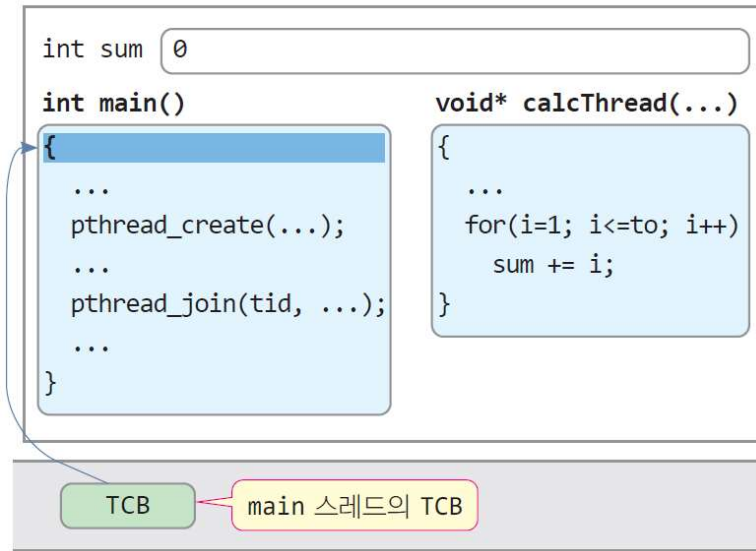
    pthread_join(tid, NULL); // tid 번호의 스레드 종료를 기다림
    printf("calcThread 스레드가 종료하였습니다.\n");
    printf("sum = %d\n", sum);
}

void* calcThread(void *param) { // param에 "100" 전달 받음
    printf("calcThread 스레드가 실행을 시작합니다.\n");
    int to = atoi(param); // to = 100
    int i;

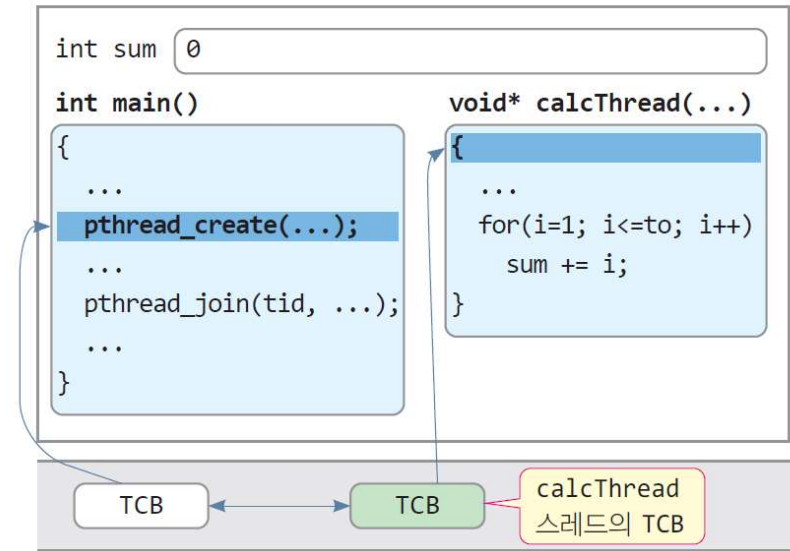
    for(i=1; i<=to; i++) // 1에서 to까지 합 계산
        sum += i; // 전역 변수 sum에 저장
}
```

스레드는 프로세스 안에 있는 실행 단위의 코드이며,
함수 형태로 작성됨을 보여주기 위한 사례

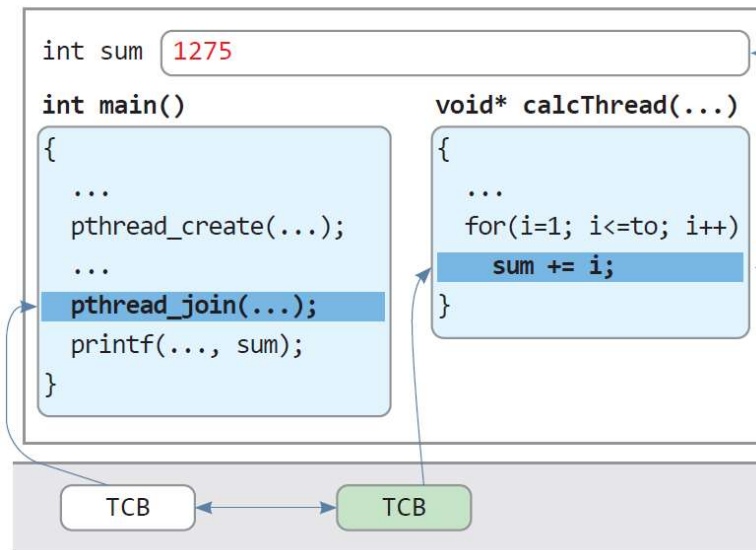
```
$ gcc -o makethread makethread.c -lpthread
$ ./makethread
calcThread 스레드가 실행을 시작합니다.
calcThread 스레드가 종료하였습니다.
sum = 5050
$
```



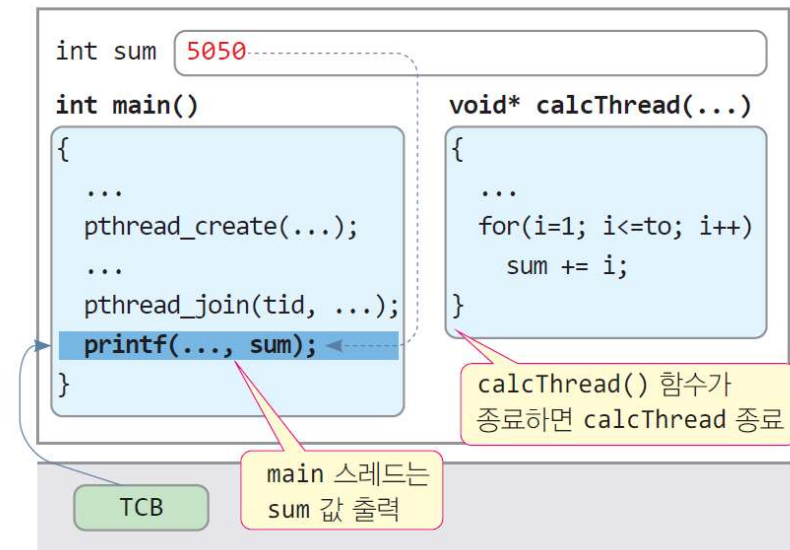
(1) 프로그램 실행 시작. main 스레드(TCB) 자동 생성



(2) main 스레드가 calcThread 생성. 두 스레드가 번갈아 실행. 스레드의 실행 순서는 알 수 없음



(3) main 스레드는 calcThread 스레드의 종료를 기다리고 있고, calcThread는 for 문을 실행하여 현재 1에서 50까지 합을 sum에 저장한 상태



(4) calcThread() 함수가 종료하면 calcThread 스레드가 종료되고 calcThread의 TCB 제거. main 스레드는 sum 값 5050을 화면에 출력

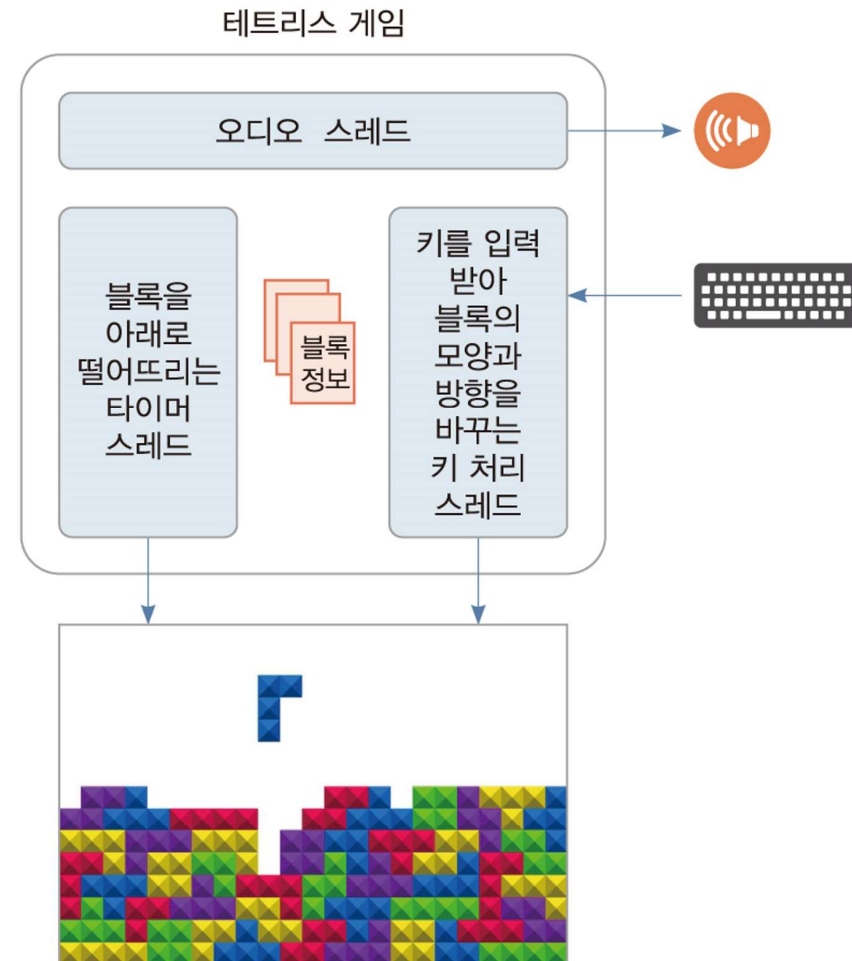
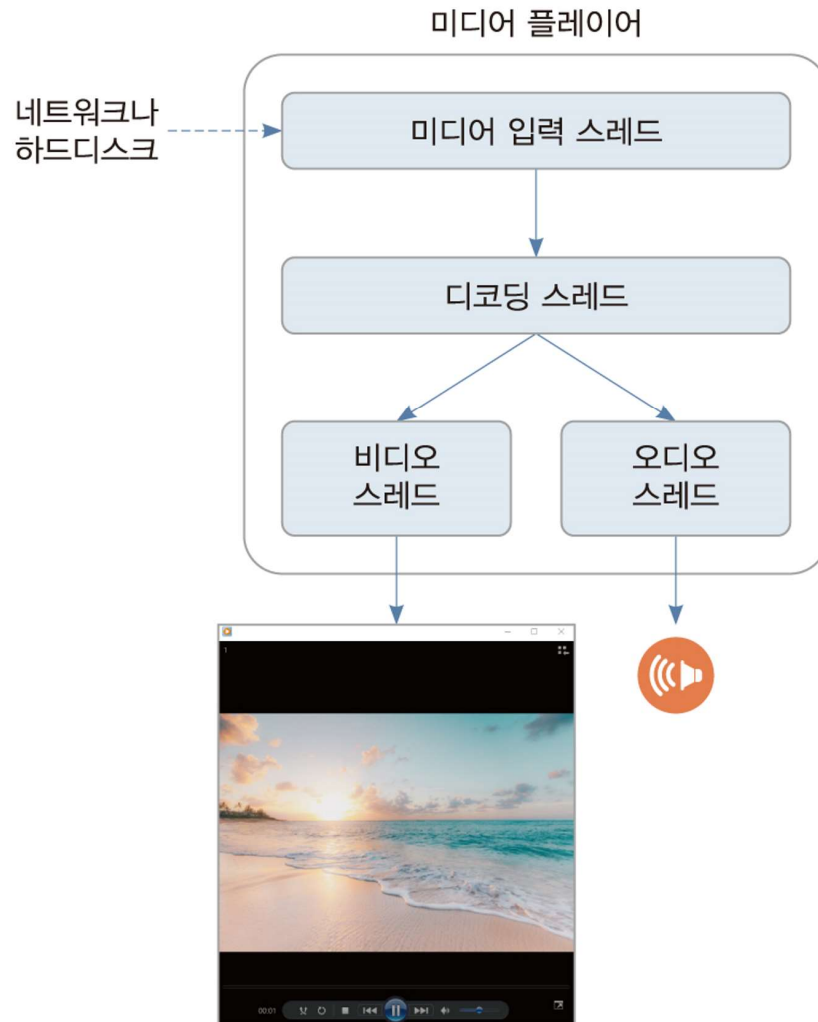
맛보기 프로그램을 통한 스레드에 대한 이해

16

- 프로세스가 생성되면 자동으로 main 스레드 생성
 - ▣ main 스레드는 main() 함수 실행
- 스레드 코드는 함수로 만들어진다.
 - ▣ calcThread() 함수
- 스레드 생성
 - ▣ 스레드는 pthread_create() 등 라이브러리 함수나 시스템 호출을 통해 생성
- 스레드마다 TCB 1개 생성
 - ▣ TCB에는 스레드의 시작 주소 저장. 이 주소에서 실행 시작
 - 이 주소를 CPU로 옮기는 컨텍스트 스위칭이 일어나면, CPU가 스레드 코드 실행
 - ▣ 컨텍스트 스위칭되어 스레드가 중단되면 TCB에 현재 컨텍스트 정보 저장(실행을 재개할 주소 저장)
 - ▣ TCB의 존재를 스레드의 존재로 인식하면 됨
- 스레드는 스케줄링되고 실행되는 실행 단위
- 프로세스는 스레드들의 컨테이너
 - ▣ 프로세스는 더 이상 실행 단위 아님
 - ▣ 맛보기 프로세스 내에 main 스레드와 calcThread 있음
 - ▣ 두 스레드가 종료되어야 프로세스 종료
- 프로세스는 스레드들에게 공유 공간 제공
 - ▣ 프로세스의 코드와 전역 변수는 모든 스레드에 의해 공유
 - ▣ 사례에서 main 스레드와 calcThread 스레드는 sum 변수 공유

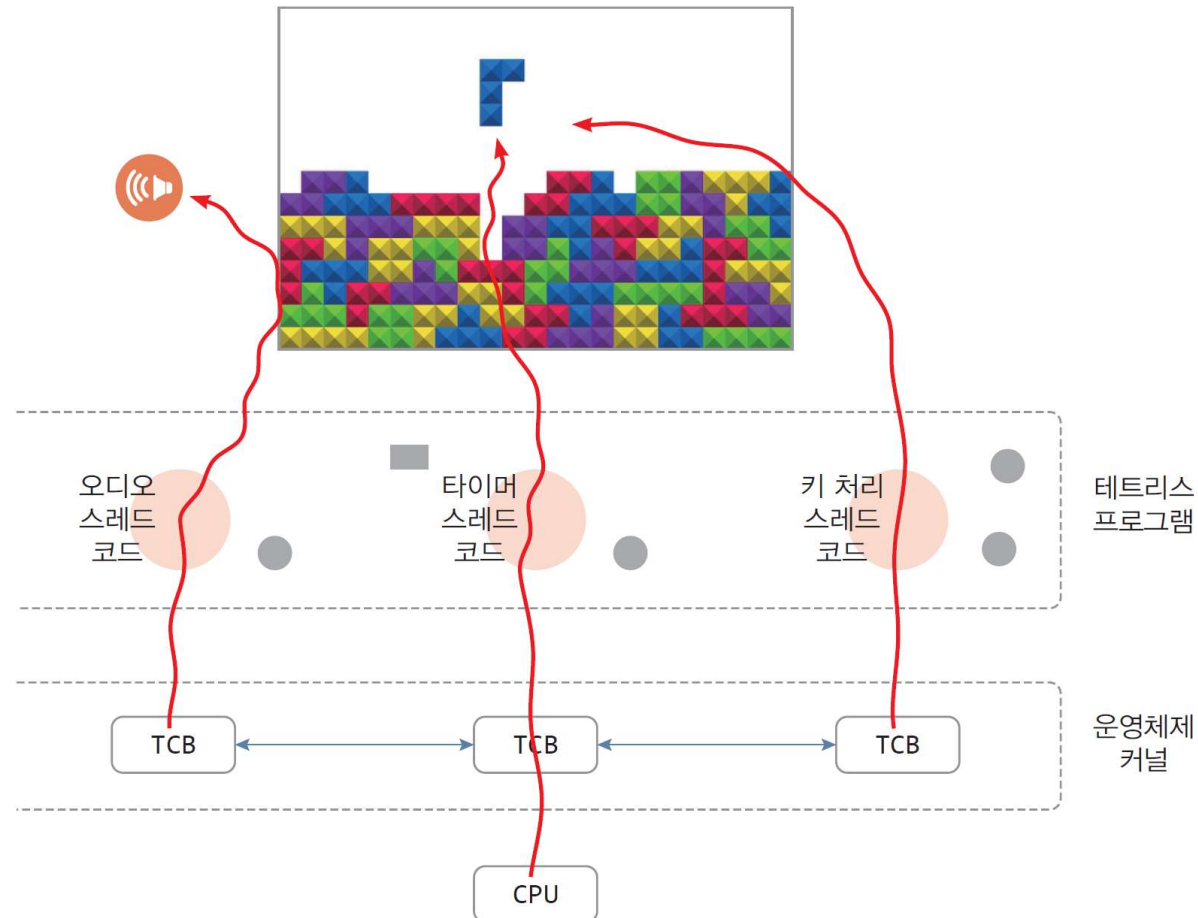
멀티스레드 응용프로그램 사례

17



멀티스레딩 분석

18



* 함수는 다른 함수에 의해 호출되어 실행되지만,
스레드 함수의 코드는 CPU가 바로 실행하도록 커널에 의해 제어됨

TIP. 멀티스레딩과 concurrency, parallelism

19

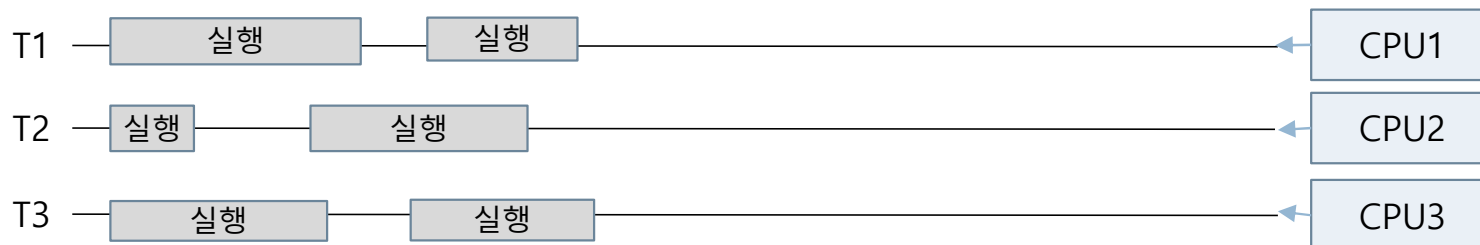
□ concurrency(동시성)

- 1개의 CPU에서 2개 이상의 스레드가 동시에 실행 중인 상태
 - 스레드가 입출력으로 실행이 중단될 때 다른 스레드 실행
 - 타임 슬라이스 단위로 CPU를 사용하도록 번갈아 스레드 실행
- concurrency 사례 - 3개의 스레드가 1개 CPU에 의해 동시 실행



□ parallelism(병렬성)

- 2개 이상의 스레드가 다른 CPU에서 같은 시간에 동시 실행
- parallelism 사례 - 3개의 스레드가 3개의 CPU에 의해 동시 실행



3. 스레드 주소 공간과 컨텍스트

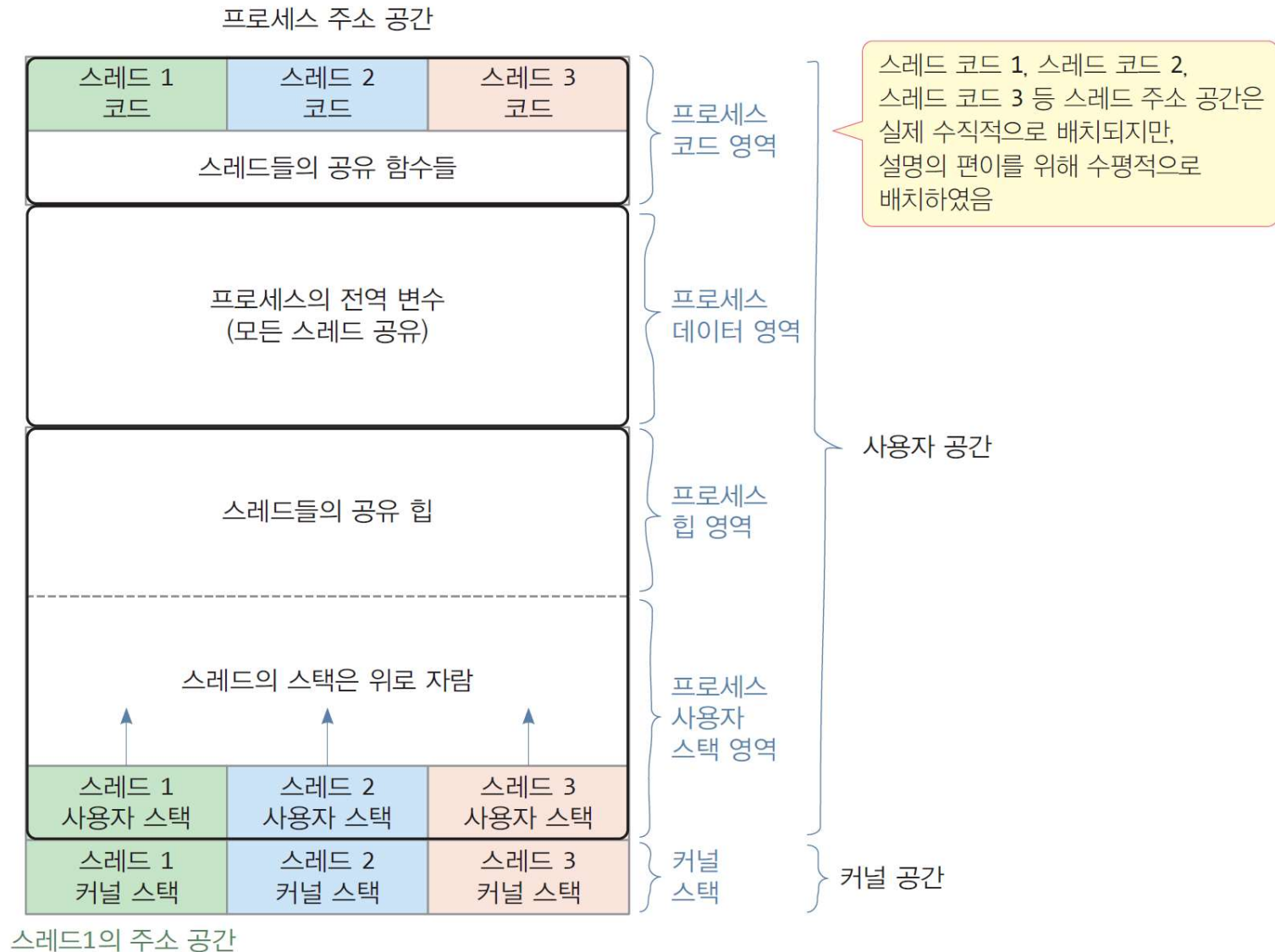
스레드 주소 공간

21

- 스레드 주소 공간
 - ▣ 스레드가 실행 중에 사용하는 메모리 공간
 - 스레드의 코드, 데이터, 힙, 스택 영역
 - ▣ 프로세스의 주소 공간 내에 형성
- 스레드 주소 공간은 프로세스 주소 공간 내에서 사적 공간과 공유 공간으로 구분
 - ▣ 스레드 사적 공간
 - 스레드 스택
 - 스레드 로컬 스토리지(TLS, Thread local storage)
 - ▣ 스레드 사이의 공유 공간
 - 프로세스의 코드(스레드 코드 포함)
 - 프로세스의 데이터 공간
 - 프로세스의 힙 영역

스레드 주소 공간 사례

22



* 참고
스레드 로컬 스토리지(TLS)
는 운영체제에 따라
힙이나 스택에 할당됨.

※ 스레드의 주소 공간은 프로세스 주소 공간 내에 존재한다.

스레드 주소 공간에 대한 설명(1)

23

- 스레드 코드 영역
 - ▣ 스레드가 실행할 작업의 함수, 프로세스의 코드 영역에 있음
 - ▣ 스레드는 프로세스의 코드 영역에 있는 다른 모든 함수 호출 가능
- 스레드 데이터 영역
 - ▣ 스레드가 사용할 수 있는 데이터 공간
 - ▣ 2개의 공간
 - 프로세스에 선언된 모든 전역 변수들 - 프로세스의 데이터 영역
 - 모든 스레드에 의해 공유되는 공간
 - 스레드들 사이의 통신 공간으로 유용하게 사용
 - 개별 스레드의 전용 변수 공간(스레드 로컬 스토리지)
 - 각 스레드마다 독립된 전용 변수 공간
 - `static __thread`와 같은 특별한 키워드로 선언
 - 운영체제에 따라 프로세스의 힙이나 스택에 할당됨
- 스레드 힙
 - ▣ 모든 스레드가 동적 할당받는 공간, 프로세스 힙 공간을 공유하여 사용
 - ▣ 스레드에서 `malloc()`를 호출하면 프로세스의 힙 공간에서 메모리 할당
- 스레드 스택
 - ▣ 스레드가 생성될 때,
 - 프로세스에게 할당된 스택에서 사용자 스택 할당
 - 커널 공간에 스레드마다 커널 스택 할당
 - ▣ 스레드가 시스템 호출로 커널에 진입할 때, 커널 스택 활용
 - ▣ 스레드 종료 시, 스레드가 할당 받은 사용자 스택과 커널 스택 반환

스레드 주소 공간에 대한 설명(2)

24

- 스레드 로컬 스토리지(TLS, Thread Local Storage)
 - ▣ **스레드마다** 안전하게 다루고자 하는 데이터를 저장하기 위한 **별도의 영역**
 - 프로세스의 데이터 영역은 모든 스레드의 공용 공간이므로
 - ▣ 스레드가 자신만 사용할 변수들을 선언할 수 있는 영역
 - ▣ 생성되는 영역 : 운영체제마다 다름. 대체로 힙이나 **스택**에 할당
 - ▣ 프로그램에서 할당받는 방법
 - 프로그래밍 언어마다 다름
- C 언어에서 스레드 로컬 스토리지 할당 사례
 - ▣ 스레드 로컬 스토리지가 선언되면, 스레드가 생성될 때마다 모든 스레드에 동일한 크기의 공간 할당

```
static __thread int sum = 5; // 스레드 로컬 스토리지에 할당될 변수 sum 선언
```

- 스레드가 생길 때마다, 각 스레드에 sum 변수 크기의 스레드 로컬 스토리지가 할당되고 초기값이 5로 설정
- ▣ 스레드가 종료하면 스레드의 로컬 스토리지 사라짐

스레드 주소 공간 확인 - 스레드 코드, TLS, 프로세스 전역 변수, 지역 변수, 사용자 스택

25

아래 코드를 보고 스레드 주소 공간을 그려라.

makethreadwithTLS.c

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

void printsum(); // 모든 스레드에 의해 호출되는 함수
void* calcThread(void *param); // 스레드 코드(함수)

static __thread int tsum = 5; // 스레드 로컬 스토리지(TLS)에
int total = 0; // 프로세스의 전역 변수, 모든 스레드에 의해 공유

int main() {
    char *p[2] = {"100", "200"};
    int i;
    pthread_t tid[2]; // 스레드의 id를 저장할 정수 배열
    pthread_attr_t attr[2]; // 스레드 정보를 담을 구조체

    // 2개의 스레드 생성
    for(i=0; i<2; i++) {
        pthread_attr_init(&attr[i]); // 구조체 초기화
        pthread_create(&tid[i], &attr[i], calcThread, p[i]); // 스레드 생성
        printf("calcThread 스레드가 생성되었습니다.\n");
    }

    // 2개 스레드의 종료를 기다린 후에 total 값 출력
    for(i=0; i<2; i++) {
        pthread_join(tid[i], NULL); // 스레드 tid[i]의 종료대기
        printf("calcThread 스레드가 종료하였습니다.\n");
    }
    printf("total = %d\n", total); // 2개 스레드의 합이 누적된 total 출력
    return 0;
}
```

```
void* calcThread(void *param) { // 스레드 코드
    printf("스레드 생성 초기 tsum = %d\n", tsum); // TLS 변수 tsum의 초기값 출력

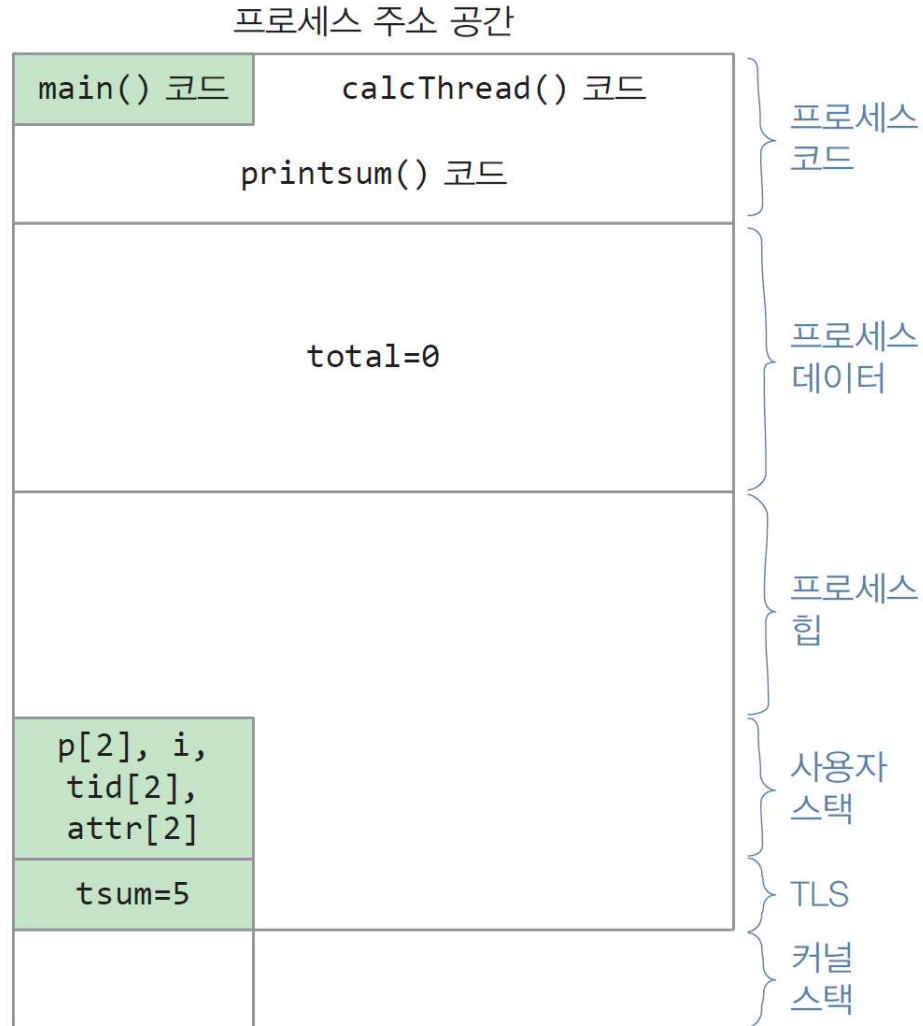
    int i, sum = 0; // 지역 변수
    for(i=1; i<= atoi(param); i++) sum += i; // 1~param까지 더하기

    tsum = sum; // TLS 변수 tsum에 합 저장
    printsum();
    total+=sum; // 전역 변수 total에 합 누적
}

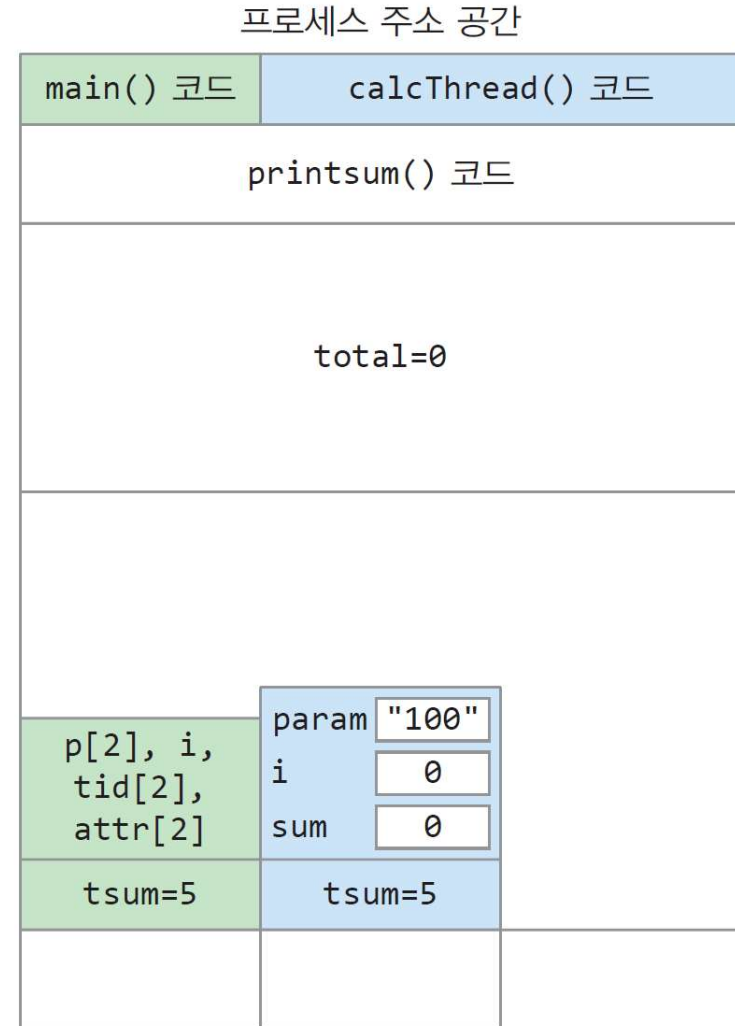
void printsum() { // 모든 스레드가 호출할 수 있는 공유 함수
    printf("계산 후 tsum = %d\n", tsum);
}
```

```
$ gcc -o mtTLS makethreadwithTLS.c -lpthread
$ ./mtTLS
calcThread 스레드가 생성되었습니다.
스레드 생성 초기 tsum = 5
calcThread 스레드가 생성되었습니다.
계산 후 tsum = 5050
스레드 생성 초기 tsum = 5
계산 후 tsum = 20100
calcThread 스레드가 종료하였습니다.
calcThread 스레드가 종료하였습니다.
total = 25150
$
```

사례의 스레드 주소 공간(1)



(1) 프로세스가 적재되고, main 스레드가 생성되어 main 스레드의 주소 공간 형성



(2) 첫 번째 calcThread 스레드가 생성되어 프로세스 내에 주소 공간 형성

사례의 스레드 주소 공간(2)

프로세스 주소 공간

main() 코드	calcThread() 코드	
printsum() 코드		
total=5050		
p[2], i, tid[2], attr[2]	param "100"	param "200"
	i 100	i 9
	sum 5050	sum 45
tsum=5	tsum=5050	tsum=5

- (3) 2번째 calcThread 스레드가 생성되어 주소 공간을 형성하고, 첫 번째 calcThread가 1에서 100까지 합을 구한 상태. 그리고 2번째 calcThread 스레드는 1에서 9까지밖에 합을 구하지 못한 상태

프로세스 주소 공간

main() 코드	calcThread() 코드	
printsum() 코드		
total=25150		
p[2], i, tid[2], attr[2]	param	param "200"
	i	i 200
	sum	sum 20100
tsum=5	tsum=5050	tsum=20100

- (4) 첫 번째 calcThread 스레드가 종료하여 TLS를 포함하여 스레드의 사용자 스택과 커널 스택이 소멸되고, 2번째 calcThread 스레드가 1에서 200까지의 합을 구한 상태

사례에 대한 질문

28

Q1. 이 프로그램이 실행되는 동안 총 몇 개의 스레드가 실행되는가?

A. main 스레드를 포함하여 총 3개의 스레드가 실행된다. main 스레드를 생각 못하고 2개의 스레드가 실행된다고 답하면 안 됨

Q2. tsum을 어떤 변수라고 부르는가? 그리고 total 변수와의 차이점은 무엇인가?

A. tsum은 TLS 변수로 불린다. tsum은 각 스레드의 TLS 영역에 스레드마다 생기고 스레드에 의해 사적으로 사용. total은 프로세스의 전역변수로서 프로세스 전체에 하나만 생기고 프로세스에 속한 모든 스레드에 의해 공유

Q3. 이 프로그램이 실행되는 동안 프로세스와 스레드의 주소 공간을 그려라.

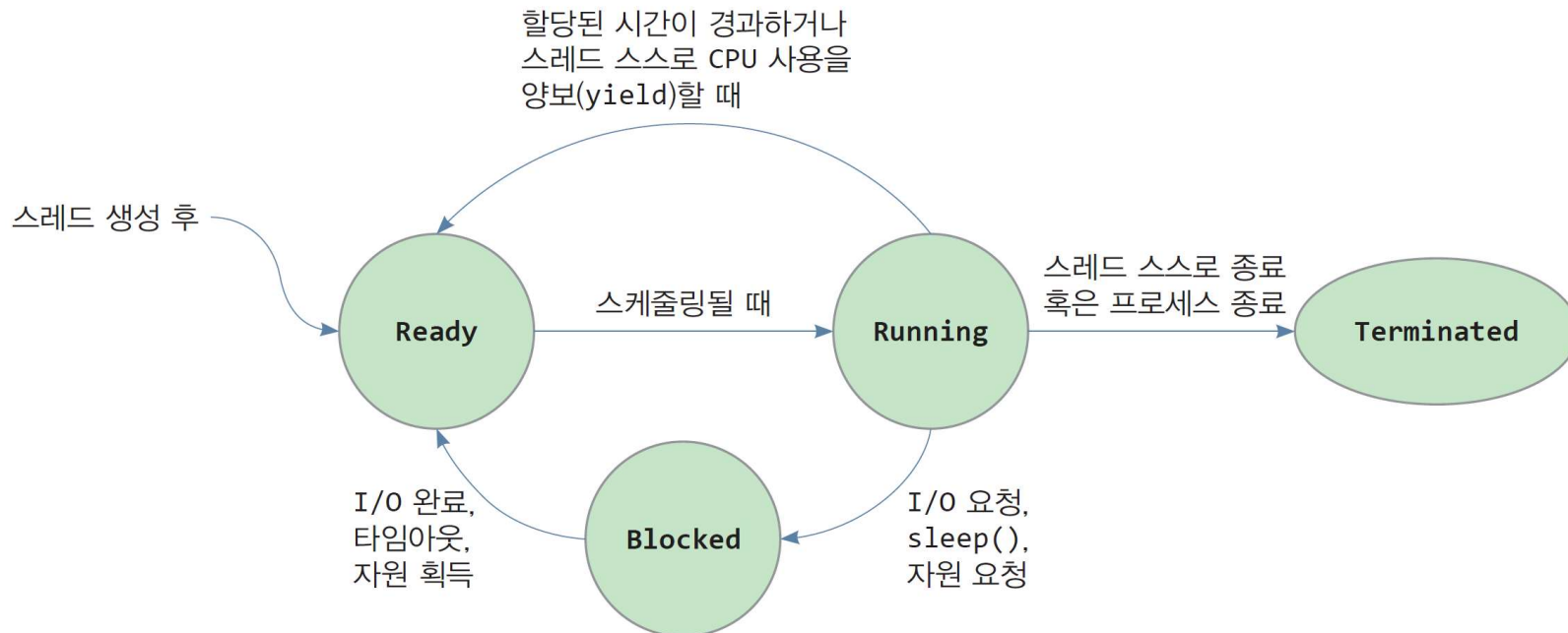
A. 그림 참고

스레드 상태

29

□ 스레드 일생

- 스레드는 생성, 실행, 중단, 실행, 소멸의 여러 상태를 거치면서 실행
- 스레드 상태는 TCB에 저장
- 스레드 상태
 - 준비 상태(Ready) - 스레드가 스케줄 되기를 기다리는 상태
 - 실행 상태(Running) - 스레드가 CPU에 의해 실행 중인 상태
 - 대기 상태(Blocked) - 스레드가 입출력을 요청하거나 sleep() 같은 시스템 호출로 인해 중단된 상태
 - 종료 상태(Terminated) - 스레드가 종료한 상태



스레드 운용(operation)

30

□ 응용프로그램이 스레드에 대해 할 수 있는 운용의 종류

1) 스레드 생성

- 프로세스가 생성되면 운영체제에 의해 자동으로 main 스레드 생성
- 스레드는 시스템 호출이나 라이브러리 함수를 호출하여 새 스레드 생성 가능

2) 스레드 종료

- 프로세스 종료와 스레드 종로의 구분 필요
- 프로세스 종료
 - 프로세스에 속한 아무 스레드가 `exit()` 시스템 호출을 부르면 프로세스 종료(모든 스레드 종료)
 - 메인 스레드의 종료(C 프로그램에서 `main()` 함수 종료) – 모든 스레드가 함께 종료
 - 모든 스레드가 종료하면 프로세스 종료
- 스레드 종료
 - `pthread_exit()`와 같이 스레드만 종료하는 함수 호출 시 해당 스레드만 종료
 - `main()` 함수에서 `pthread_exit()`을 부르면 main 스레드만 종료

3) 스레드 조인

- 스레드가 다른 스레드가 종료할 때까지 대기
 - 주로 부모 스레드가 자식 스레드의 종료 대기

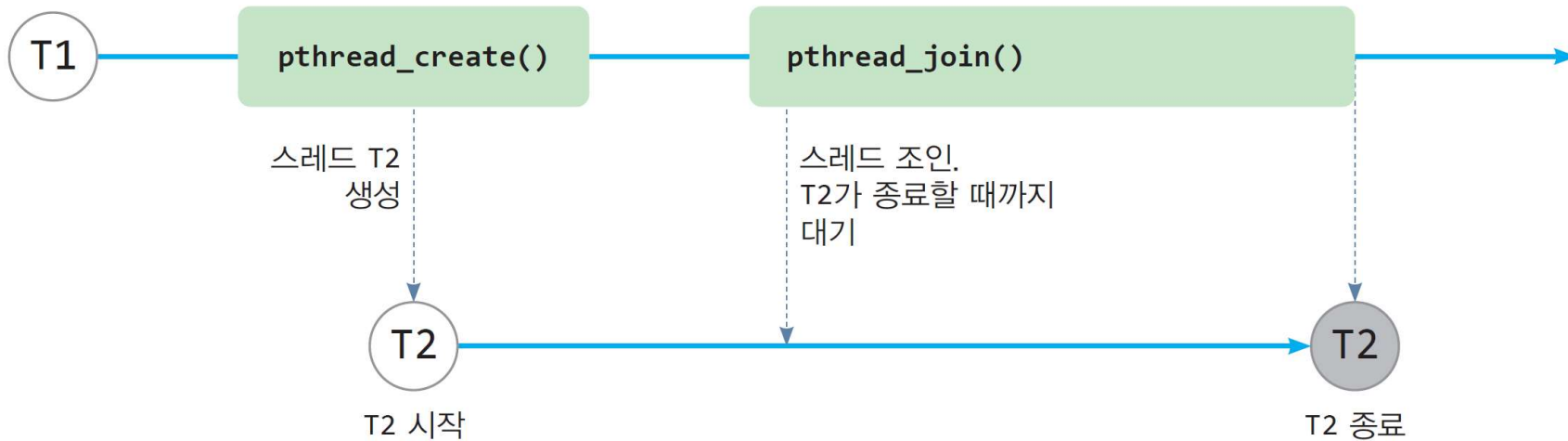
4) 스레드 양보

- 스레드가 자발적으로 `yield()`와 같은 함수 호출을 통해 스스로 실행을 중단하고 다른 스레드를 스케줄하도록 요청

스레드 조인

31

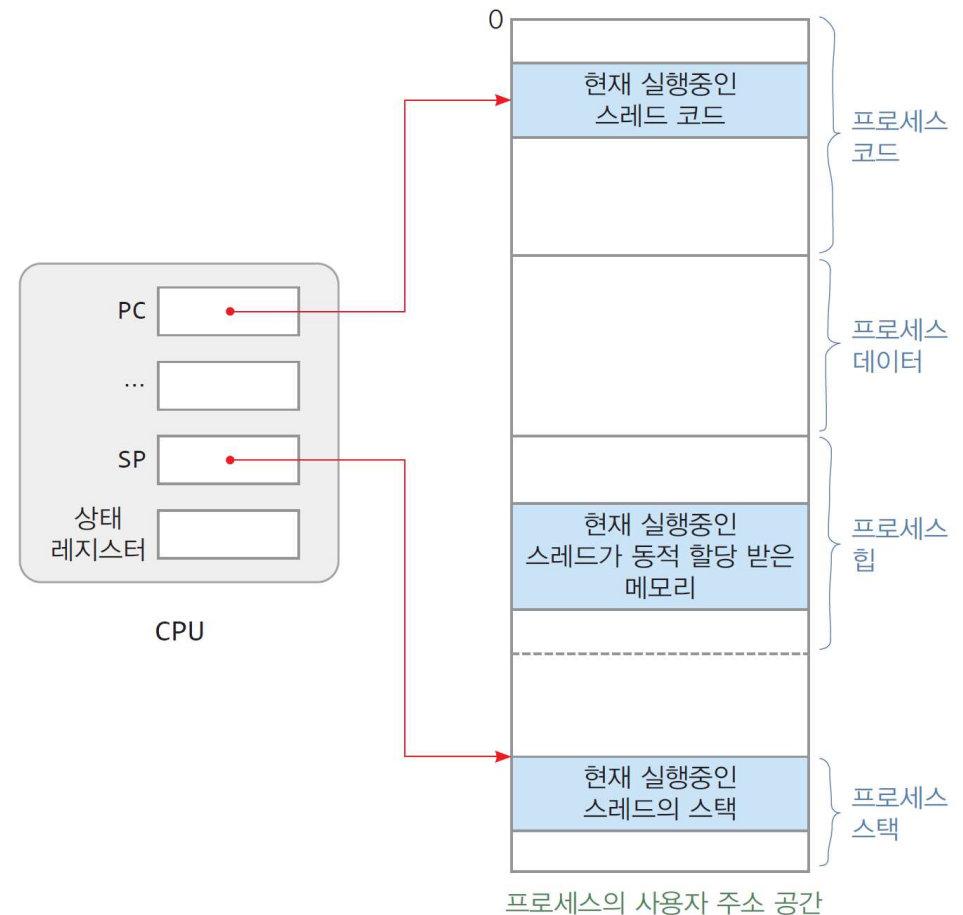
- 스레드가 다른 스레드의 종료를 기다리는 행위



스레드 컨텍스트

32

- 스레드 컨텍스트
 - ▣ 스레드가 현재 실행중인 일체의 상황
 - ▣ CPU 레지스터 값들
- 스레드 컨텍스트 정보
 - ▣ PC 레지스터
 - 실행 중인 코드 주소
 - ▣ SP 레지스터
 - 실행 중인 함수의 스택 주소
 - ▣ 상태 레지스터
 - 현재 CPU의 상태 정보
 - ▣ CPU에 기타 수십 개의 레지스터 있음
 - 데이터 레지스터 등
 - ▣ 컨텍스트 스위치 될 때 TCB에 저장



* 스레드 컨텍스트를 저장해 두었다가 CPU에 복귀시키면,
이전에 중단된 상태에서 실행할 수 있음

스레드 제어 블록

33

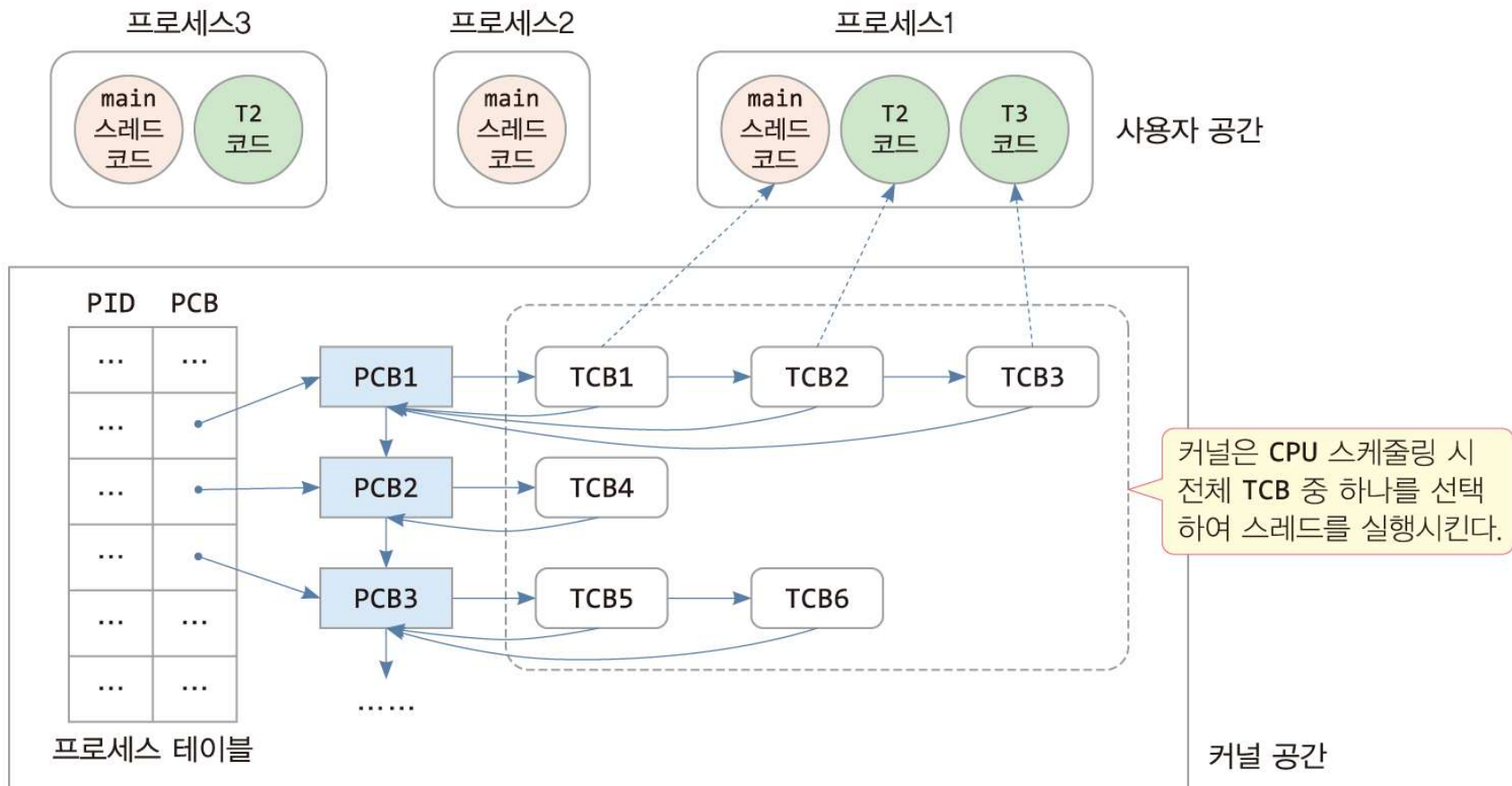
- 스레드 제어 블록, TCB(Thread Control Block)
 - ▣ 스레드를 실행 단위로 다루기 위해 스레드에 관한 정보를 담은 구조체
 - 스레드 엔터티(thread entity), 스케줄링 엔터티(scheduling entity)라고도 불림
 - ▣ 커널 영역에 만들어지고, 커널에 의해 관리
 - 스레드가 생성될 때 커널에 의해 만들어지고, 스레드가 소멸되면 사라짐

구분	요소	설명
스레드 정보	tid	스레드 ID. 스레드가 생성될 때 부여된 고유 번호
	state	스레드의 상태 정보, 실행(Running), 준비(Ready), 블록(Blocked), 종료(Terminated)
컨텍스트	PC	CPU의 PC 레지스터 값. 스레드가 스케줄될 때 실행할 명령의 주소
	SP	CPU의 SP 레지스터 값. 스레드 스택의 톱 주소
	다른 레지스터들	스레드가 중지될 때의 여러 CPU 레지스터 값들
스케줄링	우선순위	스케줄링 우선순위
	CPU 사용 시간	스레드가 생성된 이후 CPU 사용 시간
관리를 위한 포인터들	PCB 주소	스레드가 속한 프로세스 제어 블록(PCB)에 대한 주소
	다른 TCB에 대한 주소	프로세스 내 다른 TCB들을 연결하기 위한 링크
	블록 리스트/준비 리스트 등	입출력을 대기하고 있는 스레드들을 연결하는 TCB 링크, 준비 상태에 있는 스레드들을 연결하는 TCB 링크(스레드 스케줄링 시 사용) 등

스레드와 TCB, 그리고 PCB와의 관계

34

- 프로세스 : 스레드들이 생기고 활동하는 자원의 컨테이너
- TCB들은 링크드 리스트로 연결



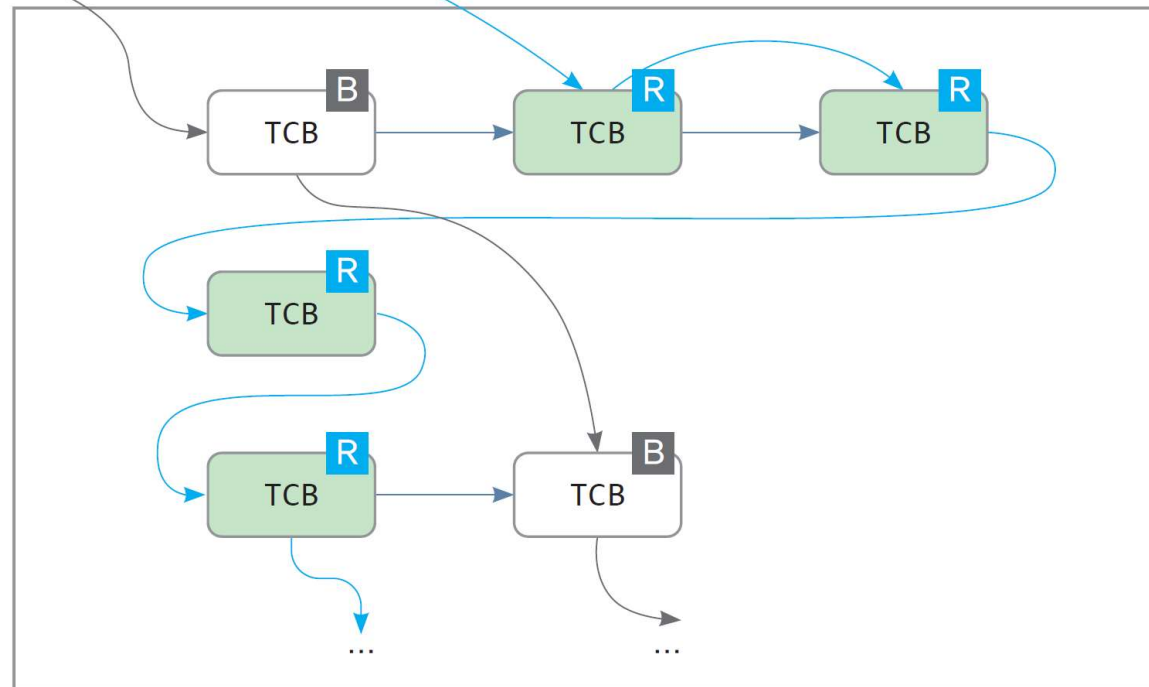
준비 리스트와 블록 리스트

35

- 준비 리스트
 - ▣ 준비 상태에 있는 스레드들의 TCB를 연결하는 링크드 리스트
 - ▣ 스레드 스케줄링은 준비 리스트의 TCB들 중 하나 선택
- 블록 리스트
 - ▣ 블록 상태에 있는 스레드들의 TCB를 연결하는 링크드 리스트

준비 리스트

블록 리스트



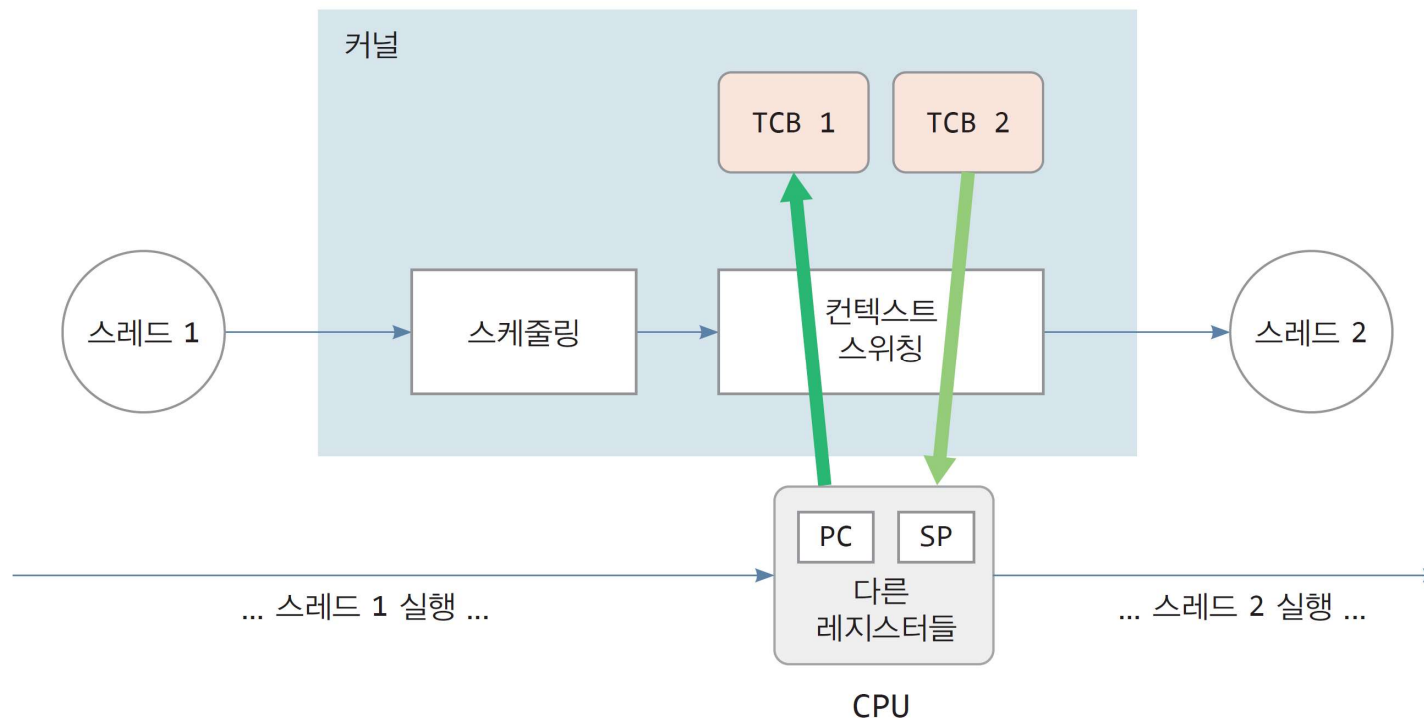
R Ready 상태

B Blocked 상태

스레드 컨텍스트 스위칭

36

- 스레드 컨텍스트 스위칭(스레드 스위칭)
 - 스레드 스케줄링 후,
 - 현재 실행중인 스레드를 중단시키고, 선택된 스레드에게 CPU 할당
 - 현재 CPU 컨텍스트를 TCB에 저장하고,
 - 선택된 스레드의 TCB에서 컨텍스트를 CPU에 적재, CPU는 선택된 스레드 실행



스레드 스위칭이 발생하는 4가지 경우

37

- 스레드 스위칭이 발생하는 4가지 경우
 1. 스레드가 자발적으로 다른 스레드에게 양보
 - `yield()` 등의 시스템 호출(혹은 라이브러리 호출)을 통해
 2. 스레드가 시스템 호출을 실행하여 블록되는 경우
 - `read()`, `sleep()`, `wait()` 등 I/O가 발생하거나 대기할 수 밖에 없는 경우
 3. 스레드의 타임 슬라이스(시간 할당량)를 소진한 경우
 - 타이머 인터럽트에 의해 체크되어 진행
 4. I/O 장치로부터 인터럽트가 발생한 경우
 - 현재 실행중인 스레드보다 높은 순위의 스레드가 I/O 작업을 끝낸 경우 등
- ▣ 상황에 따라 운영체제에 따라, 이들 4가지 경우 외에도 스레드 스위칭이 일어날 수도 있고, 아닐 수도 있음

스레드 스위칭이 이루어지는 위치

38

- 스레드 스위칭이 이루어지는 위치는 2가지
 1. 스레드가 시스템 호출을 하여, 커널이 **시스템 호출을 처리하는 과정에서**
 2. 인터럽트가 발생하여 **인터럽트 서비스 루틴이 실행되는 도중 커널 코드에서**

스레드 스위칭 과정(스레드 A에서 스레드 B로)

39

1) CPU 레지스터 저장 및 복귀

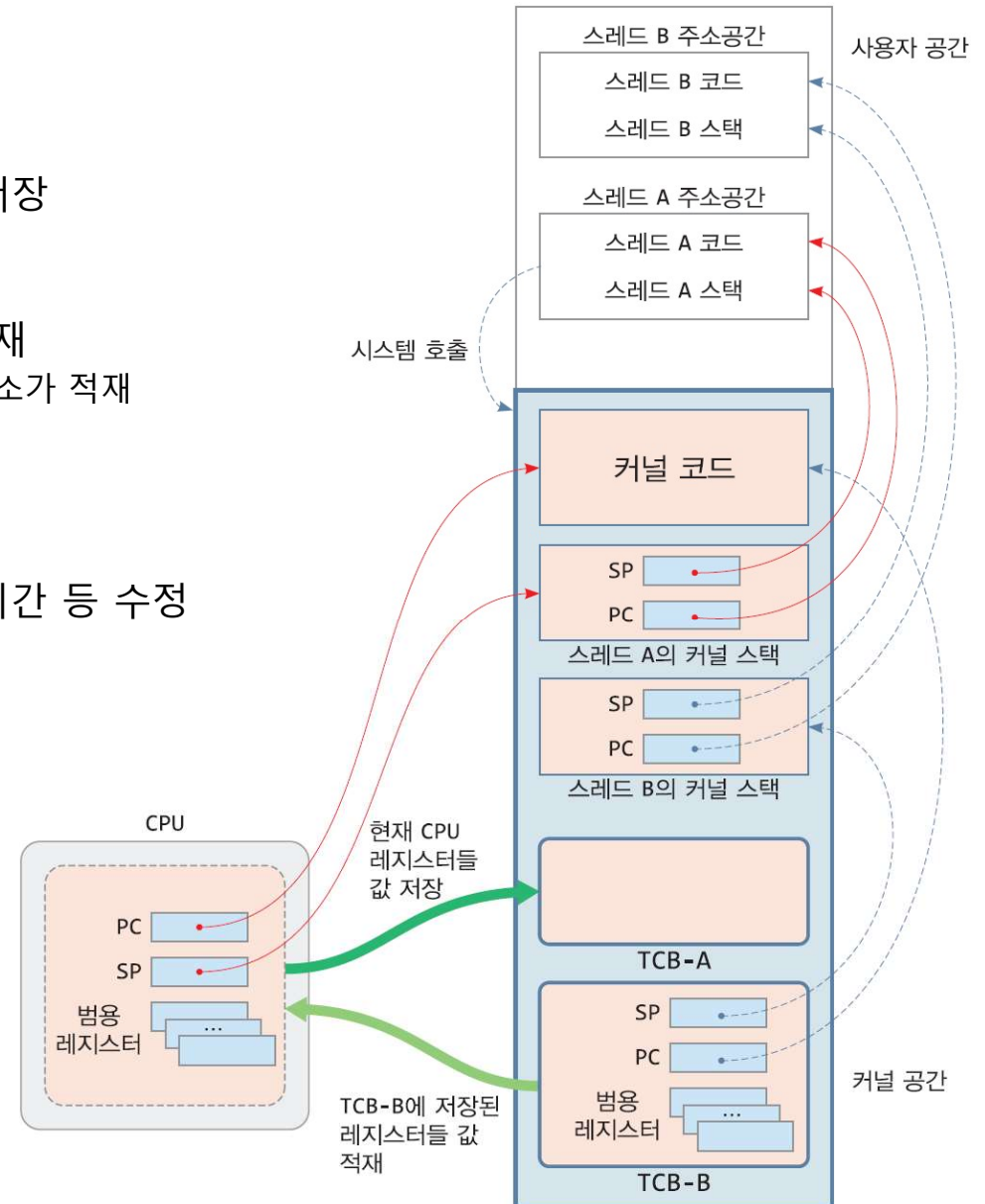
- 스레드 스위칭은 시스템 호출이나 인터럽트 서비스 도중 커널에서 진행
- 현재 실행 중인 스레드 A의 컨텍스트를 TCB-A에 저장
 - 현재 CPU의 PC는 커널 코드를,
 - SP는 스레드 A의 커널 스택을 가리킴
- TCB-B에 저장된 스레드 B의 컨텍스트를 CPU에 적재
 - CPU의 PC는 스레드 B가 이전에 실행이 중단된 커널 주소가 적재
 - SP 레지스터는 스레드 B의 커널 스택을 가리킴
- CPU는 스레드 B 실행

2) 커널 정보 수정

- TCB-A와 TCB-B에 스레드 상태 정보와 CPU 사용 시간 등 수정
- TCB-A를 준비 리스트나 블록 리스트로 옮김
- TCB-B를 준비 리스트에서 분리

3) 스레드 B가 실행을 계속하면

- 스레드 B가 커널 코드에서 실행 시작
- 시스템 호출로부터 사용자 코드로 복귀할 때, 커널 스택에 저장해둔 사용자 컨텍스트를 CPU로 복귀
- 스레드 B의 사용자 코드 실행



컨텍스트 스위칭 오버헤드

40

- 컨텍스트 스위칭에는 어떤 부담(오버헤드)이 있는가?
 - ▣ 컨텍스트 스위칭은 모두 CPU 작업 -> CPU 시간 소모
 - ▣ 컨텍스트 스위칭의 시간이 길거나, 잦은 경우 컴퓨터 처리율 저하
- 구체적인 컨텍스트 스위칭 오버헤드
 - ▣ 동일한 프로세스 내 다른 스레드로 스위칭되는 경우
 - 1) 컨텍스트 저장 및 복귀
 - 현재 CPU의 컨텍스트(PC, PSP, 레지스터) TCB에 저장
 - TCB로부터 실행할 스레드의 스레드 컨텍스트를 CPU에 복귀
 - 2) TCB 리스트 조작
 - 3) 캐시 플러시와 채우기 시간
 - ▣ 다른 프로세스의 스레드로 스위칭하는 경우
 - 다른 프로세스로 교체되면, CPU가 실행하는 주소 공간이 바뀌는 큰 변화로 인한 추가적인 오버헤드 발생
 - 1) 추가적인 메모리 오버헤드
 - 시스템 내에 현재 실행 중인 프로세스의 매핑 테이블을 새로운 프로세스의 매핑 테이블로 교체
 - 2) 추가적인 캐시 오버헤드
 - 프로세스가 바뀌기 때문에, 현재 CPU 캐시에 담긴 코드와 데이터를 무효화시킴
 - 새 프로세스의 스레드가 실행을 시작하면 CPU 캐시 미스 발생, 캐시가 채워지는데 상당한 시간 소요

41

4. 커널 레벨 스레드와 사용자 레벨 스레드

커널 레벨 스레드와 사용자 레벨 스레드

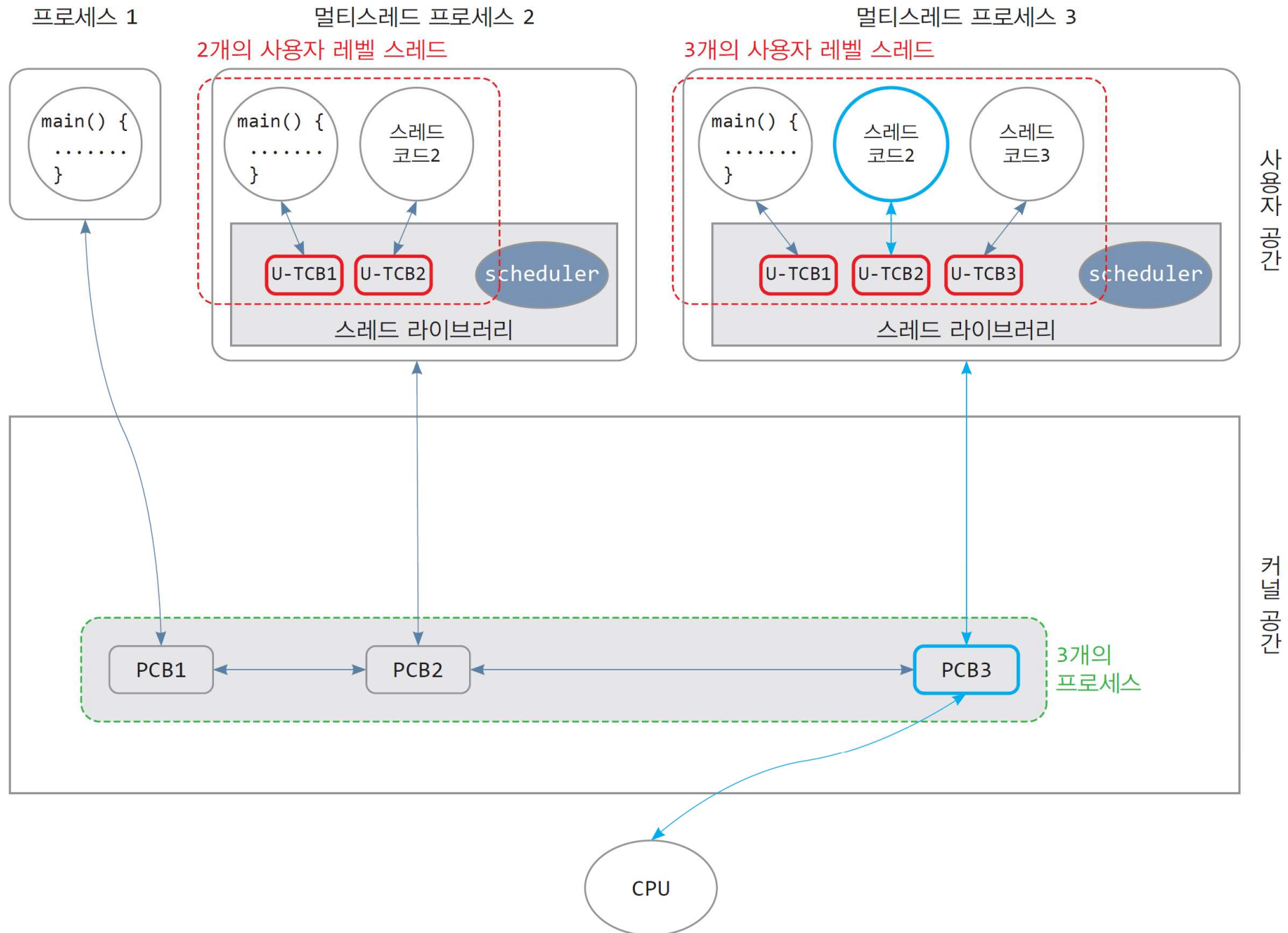
42

- 스레드 스케줄링 주체에 따라 2 종류의 스레드로 구분
 - ▣ 커널 레벨 스레드(kernel-level thread) : **커널에 의해 스케줄링되는 스레드**
 - ▣ 사용자 레벨 스레드(user-level thread) : **스레드 라이브러리에 의해 스케줄링되는 스레드**

- 커널 레벨 스레드
 - ▣ 스레드에 대한 정보(TCB)는 커널 공간에 생성되며 커널에 의해 소유됨
 - ▣ 응용프로그램이 시스템 호출을 통해 커널 레벨 스레드 생성
 - ▣ 커널이 만들고, 커널에 의해 스케줄
 - ▣ 스레드 주소 공간(스레드 코드와 데이터) : 사용자 공간에 존재
 - ▣ main 스레드는 커널 스레드
 - 응용프로그램을 적재하고 프로세스를 생성할 때 커널은 자동으로 main 스레드 생성
 - main 스레드의 TCB는 커널에 생성

- 사용자 레벨 스레드
 - ▣ 응용프로그램이 라이브러리 함수를 호출하여 사용자 레벨 스레드 생성
 - ▣ 스레드 라이브러리가 스레드 정보(U-TCB)를 사용자 공간에 생성하고 소유
 - 스레드 라이브러리는 사용자 공간에 존재
 - 커널은 사용자 레벨 스레드의 존재에 대해 알지 못함
 - ▣ 스레드 라이브러리에 의해 스케줄
 - ▣ 스레드 주소 공간(스레드 코드와 데이터) : 사용자 공간에 존재

프로세스 기반의 운영체제에서 스레드 라이브러리에 의해 관리되는 사용자 레벨 스레드

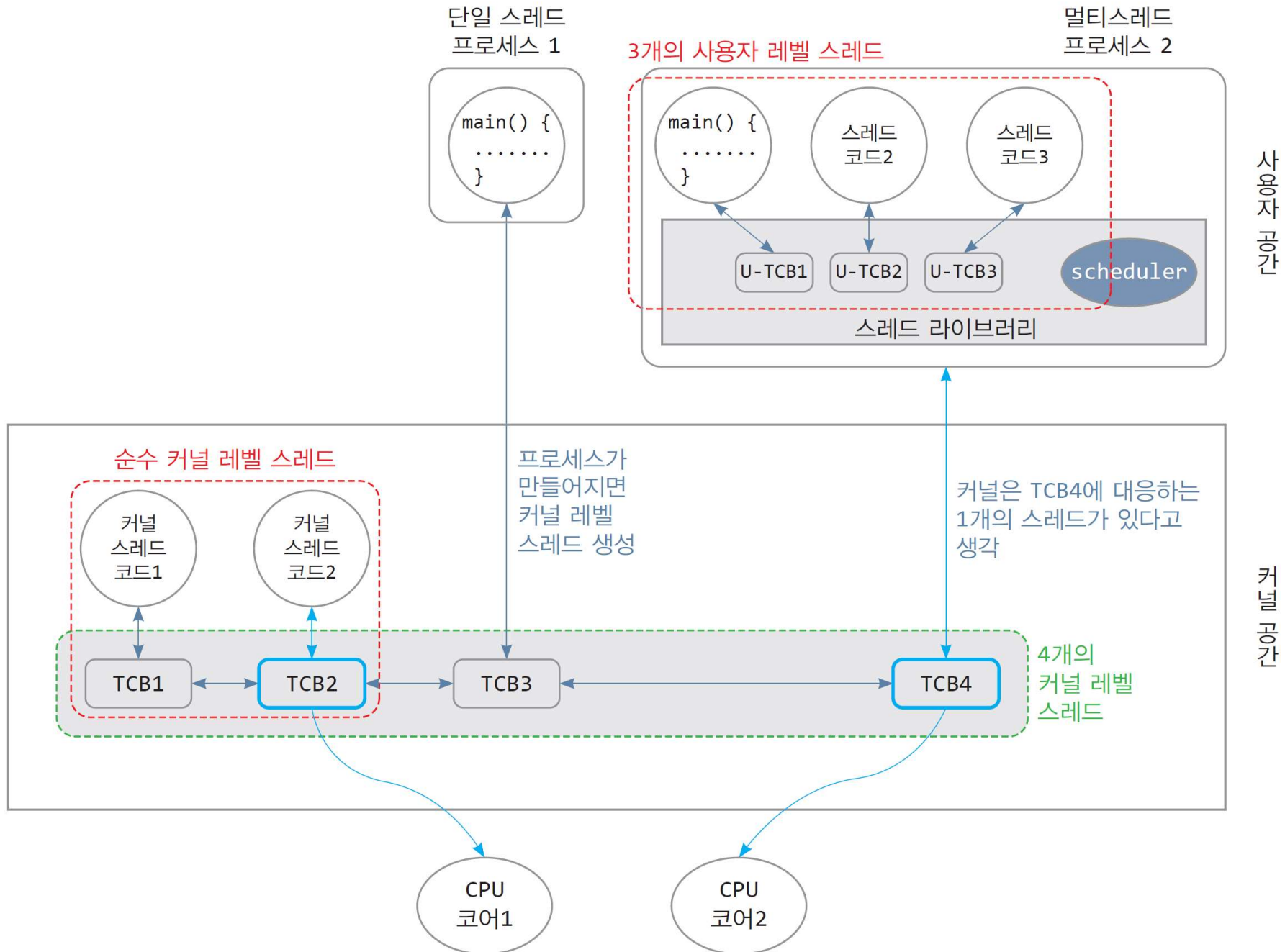


순수 커널 레벨 스레드

44

- 순수 커널 레벨 스레드(pure kernel-level thread)
 - ▣ 부팅 때부터 커널의 기능을 돕기 위해 만들어진 스레드
 - ▣ 커널 코드를 실행하는 커널 스레드
 - ▣ 스레드의 주소 공간은 모두 커널 공간에 형성
 - ▣ 커널 모드에서 작동, 사용자 모드에서 실행되는 일은 없음

커널 레벨 스레드와 사용자 레벨 스레드 사례



사례 설명(1)

46

- 사례 개요 : 프로세스 2개, 커널 레벨 스레드 4개, 사용자 레벨 스레드 3개
- 2개의 순수 커널 레벨 스레드
 - ▣ TCB1, TCB2
 - ▣ 이들 스레드의 주소 공간은 커널에 있음. 부팅 때부터 커널에서 실행되도록 만들어진 스레드
- 2개의 커널 레벨 스레드
 - ▣ 프로세스당 하나의 커널 레벨 스레드(main 스레드) 자동 생성
 - ▣ TCB3
 - 커널은 단일 스레드 프로세스1을 적재할 때 자동으로 main 스레드 TCB3 생성
 - 커널이 프로세스1을 실행시키기 위함
 - ▣ TCB4
 - 커널은 멀티스레드 프로세스2를 적재할 때 자동으로 main 스레드 TCB4 생성
 - 커널이 프로세스2를 실행시키기 위함
 - ▣ TCB3과 TCB4의 스레드 주소 공간은 모두 사용자 공간에 있음
- 3개의 사용자 레벨 스레드
 - ▣ 프로세스2의 main() 함수가 라이브러리 함수를 호출하여 자신을 사용자 레벨 스레드로 등록
 - U-TCB1 생성
 - ▣ 프로세스2의 main() 함수가 라이브러리 함수를 호출하여 2개의 사용자 레벨 스레드 추가 생성
 - U-TCB2, U-TCB3 생성

사례 설명(2)

47

□ 스레드 스케줄링

▣ 커널에 의한 스케줄(TCB1~TCB4중에서 선택)

- 코어1 : TCB2 실행(TCB2가 가리키는 커널 스레드 코드 2 실행)
- 코어2 : TCB4 실행(TCB4가 가리키는 프로세스 내의 코드 실행)
 - 처음에는 프로세스2의 `main()` 함수에서 실행을 시작하지만 현재 어떤 함수의 코드를 실행하고 있는지 알 수 없음
 - 커널은 프로세스2 내에 하나의 스레드만 있다고 생각함

□ 프로세스2에서의 사용자 스레드 스케줄링

▣ 스레드 라이브러리가 3개의 사용자 스레드 스케줄

- 예) `main()` 함수가 스레드 라이브러리의 `yield()` 함수를 호출하면, 이 함수는 스레드 라이브러리 내 scheduler를 호출하여, U-TCB2, U-TCB3 중에서 하나를 선택한다. 만일 U-TCB3이 선택되었다면, U-TCB1에 현재 실행 주소 등을 저장해 두고, U-TCB3에 저장된 실행 시작 주소(스레드 코드 3)로 점프하여 실행 시작 -> U-TCB3이 스케줄되었다.

사용자 레벨 스레드와 커널 레벨 스레드의 비교

항목	사용자 레벨 스레드	커널 레벨 스레드
정의	스레드 라이브러리에 의해 스케줄되는 스레드	커널에 의해 스케줄되는 스레드
구현	스레드 라이브러리에 의해 구현되고 다루어짐	커널에 의해 구현. 커널 API (시스템 호출) 필요
스레드 스위칭	사용자 모드에서 스레드 라이브러리에 의해 실행	커널 모드에서 커널에 의해 실행
컨텍스트 스위칭 속도	커널 레벨 스레드보다 100 배 이상 빠르다고 알려짐	커널 내에서 상당 시간 지연
멀티스레드 응용프로그램	스레드 라이브러리를 이용하여 작성하기 쉽고, 스레드 생성 속도 빠름	시스템 호출을 이용하여 스레드 생성. 스레드 생성 속도 느림
이식성(portability)	운영체제 상관없이 작성 가능하므로 높은 이식성. 스레드를 지원하지 않는 운영체제에서도 가능	스레드를 생성하고 다루는 시스템 호출이 운영체제마다 다르므로 이식성 낮음
병렬성(parallelism)	멀티 CPU 컴퓨터나 멀티 코어 CPU에서 멀티스레드의 병렬처리 안 됨	높은 병렬성. 커널 레벨 스레드들이 서로 다른 CPU나 서로 다른 코어에서 병렬 실행 가능
병렬성의 종류	concurrency (동시성)	parallelism (병렬성)
블록킹(blocking)	하나의 사용자 레벨 스레드가 시스템 호출 도중 입출력 등으로 인해 중단 (Blocked)되면 프로세스의 모든 사용자 레벨 스레드가 중단됨	하나의 커널 레벨 스레드가 시스템 호출 도중 입출력 등으로 인해 중단(Blocked)되어도 해당 스레드만 중단
커널 부담	없음	커널 코드의 실행 시간 증가. 시스템 전체에 부담
스레드 동기화	스레드 라이브러리에 의해 수행	시스템 호출을 통해 커널에 의해 수행
관리의 효율성	커널 부담 없음	커널 부담
최근 경향	멀티 코어 CPU에 적합하지 않아 줄고 있는 추세	멀티 코어 CPU에서 높은 병렬성을 얻을 수 있어 많이 사용되는 추세

5. 멀티스레드 구현

멀티스레드 구현

50

- 멀티스레드의 구현이란?
 - ▣ 응용프로그램에서 작성한 스레드가 시스템에서 실행되도록 구현하는 방법
 - 사용자가 만든 스레드가 시스템에서 스케줄되고 실행되도록 구현하는 방법
 - 스레드 라이브러리와 커널의 시스템 호출의 상호 협력 필요
- 3가지 방법
 - ▣ N:1 매핑(N개의 사용자 레벨 스레드를 1개의 커널 레벨 스레드로 매핑)
 - ▣ 1:1 매핑(1개의 사용자 레벨 스레드를 1개의 커널 레벨 스레드로 매핑)
 - ▣ N:M 매핑(N개의 사용자 레벨 스레드를 M개의 커널 레벨 스레드로 매핑)

N:1 매핑

51

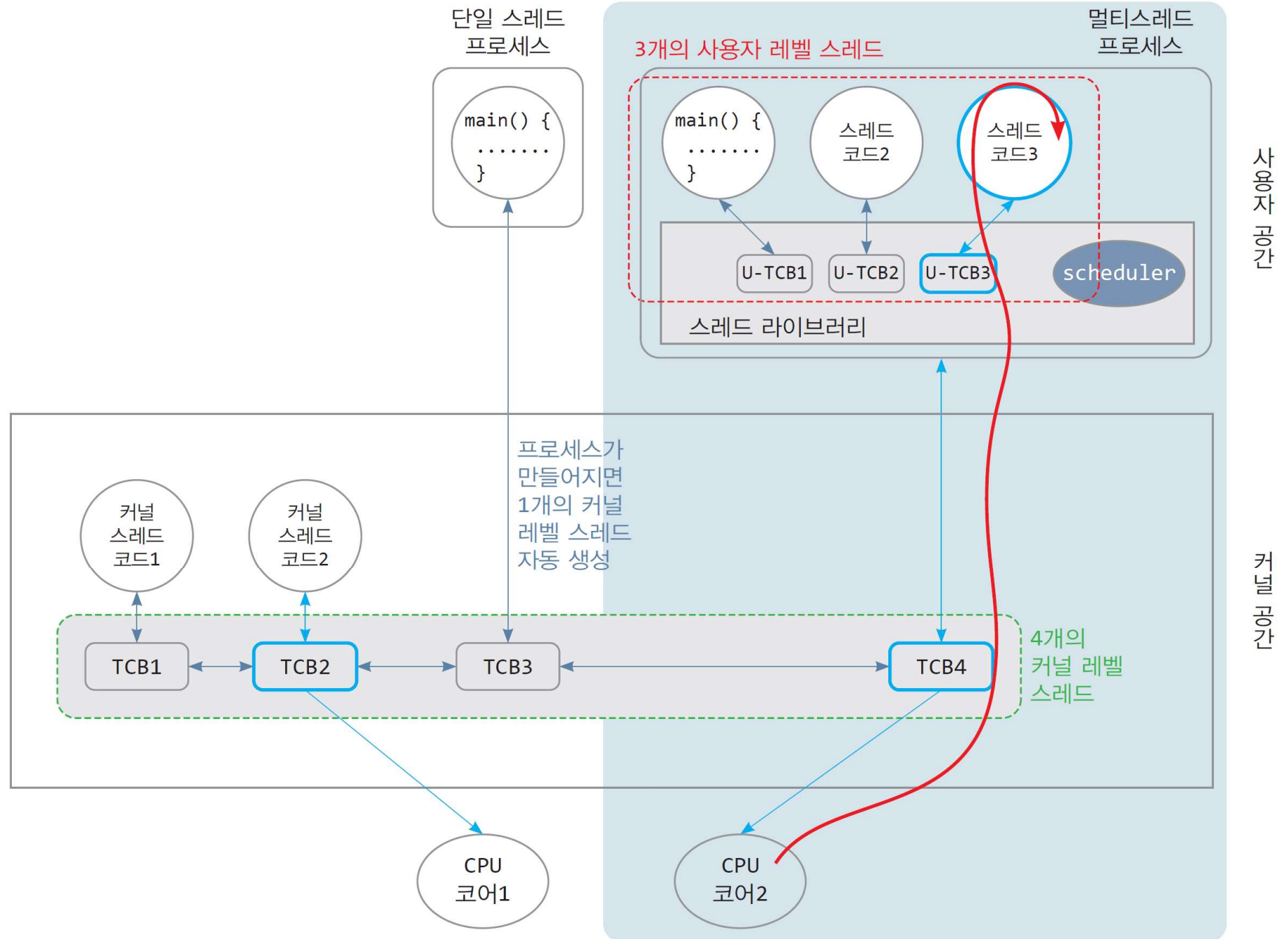
□ N:1 매핑 개념

- ▣ 모든 프로세스를 단일 스레드 프로세스로 다룸
- ▣ 커널은 프로세스 당 1개의 커널 레벨 스레드(TCB) 생성
- ▣ 프로세스의 모든 사용자 레벨 스레드(N개)가 1개의 커널 레벨 스레드에 매핑
- ▣ 사용자 레벨 스레드는 스레드 라이브러리에 의해 스케줄되고 스위칭됨

□ 매핑의 뜻

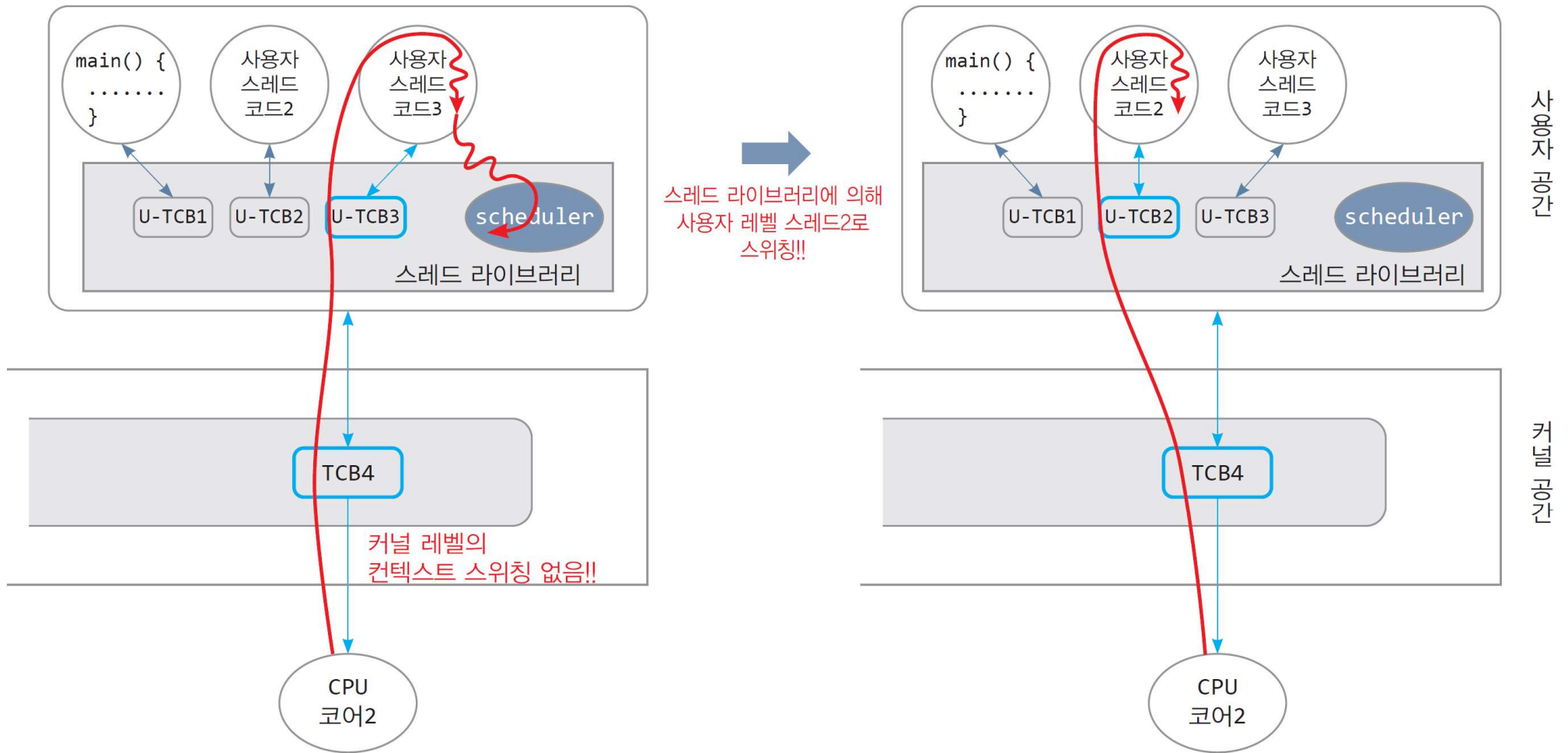
- ▣ 사용자 레벨 스레드는 해당 커널 레벨 스레드가 스케줄되어야 실행 가능하도록 묶여 있음

N:1 매핑 사례



3개의 사용자 레벨 스레드가 1개의 커널 레벨 스레드 TCB4 공유

53



(a) 사용자 레벨 스레드3 실행

(b) 사용자 레벨 스레드2 실행

N:1 매핑의 장단점

54

□ 장단점

▣ 장점

- 단일 코어 CPU에서 멀티스레드 응용프로그램의 실행 속도가 전반적으로 빠르다.
 - 스레드 생성, 스케줄, 동기화 등에 있어 커널로 진입없이 사용자 공간에서 이루어지므로

▣ 단점

- 멀티 코어 CPU가 보편화된 현대 컴퓨터에서 비효율적
 - 프로세스에 속한 여러 사용자 레벨 스레드들의 병렬 처리 안됨
- 하나의 사용자 레벨 스레드가 블록되면 프로세스 전체 블록
 - 프로세스 내 다른 사용자 레벨 스레드로 스위칭되지 못함

55



The diagram illustrates the interaction between User Space and Kernel Space during a system call.

사용자 공간 (User Space):

- main() { ... }**: The user program.
- 사용자 스레드 코드1**, **사용자 스레드 코드2**, **사용자 스레드 코드3**: User threads.
- U-TCB1**, **U-TCB2**, **U-TCB3**: User Thread Control Blocks.
- scheduler**: The thread scheduler.
- 스레드 라이브러리**: Thread library.

커널 공간 (Kernel Space):

- TCB3**, **TCB4 (Blocked)**: Kernel Thread Control Blocks.
- Blocked!!**: A state where the kernel thread is blocked.
- read() 커널 코드**: Kernel code for the read system call.
- 시스템 호출**: System call.
- CPU 코어2**: The CPU core.
- 디스크**: The disk.

Flow:

- A user thread (사용자 스레드 코드2) calls a system call (시스템 호출).
- The system call is handled by the kernel (read() 커널 코드).
- The kernel thread (TCB4) becomes blocked (Blocked!!) while waiting for data from the disk.
- The scheduler in the user space schedules another user thread (사용자 스레드 코드1) to run.
- The kernel thread (TCB3) is woken up and runs on the CPU core (CPU 코어2).
- The kernel thread (TCB3) then performs a context switch (TCB3으로 컨텍스트 스위칭!!) to the user thread (사용자 스레드 코드1).

(b) 커널은 코어2를 TCB3의 커널 레벨 스레드에게 할당

1:1 매핑

56

□ 1:1 매핑 개념

- ▣ 사용자 레벨 스레드 당 1개의 커널 레벨 스레드(TCB) 생성
- ▣ 사용자 레벨 스레드는 매핑된 커널 레벨 스레드가 스케줄될 때 실행

□ 장단점

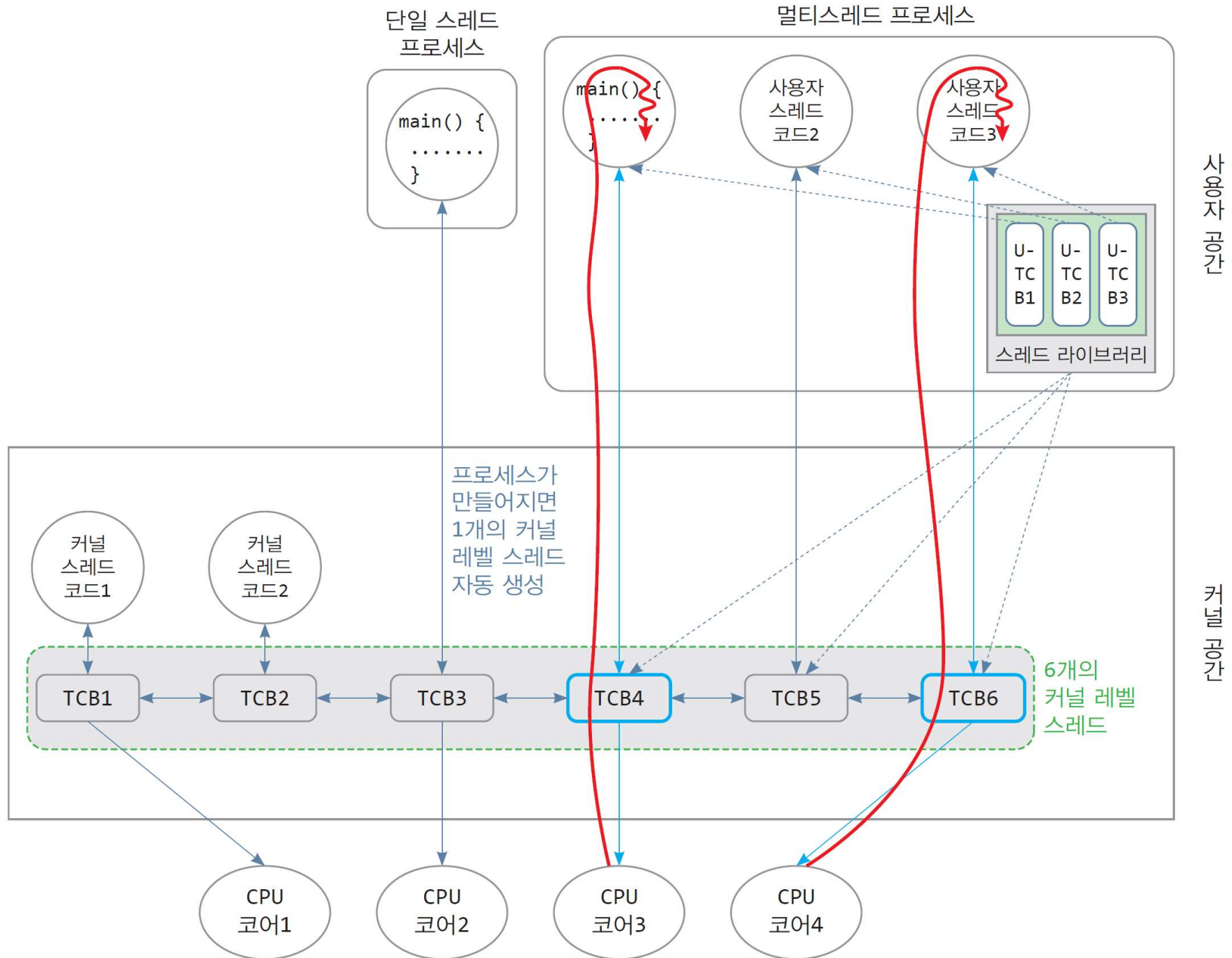
▣ 장점

- 개념이 단순하여 구현이 용이
- 멀티 코어 CPU에서 멀티스레드 응용프로그램에게 높은 병렬성 제공
- 하나의 사용자 레벨 스레드가 블록되어도 응용프로그램 전체가 블록되지 않음

▣ 단점

- 커널에게는 부담스러운 정책
- 사용자 레벨 스레드가 많아지면 커널의 부담

1:1 매핑 사례

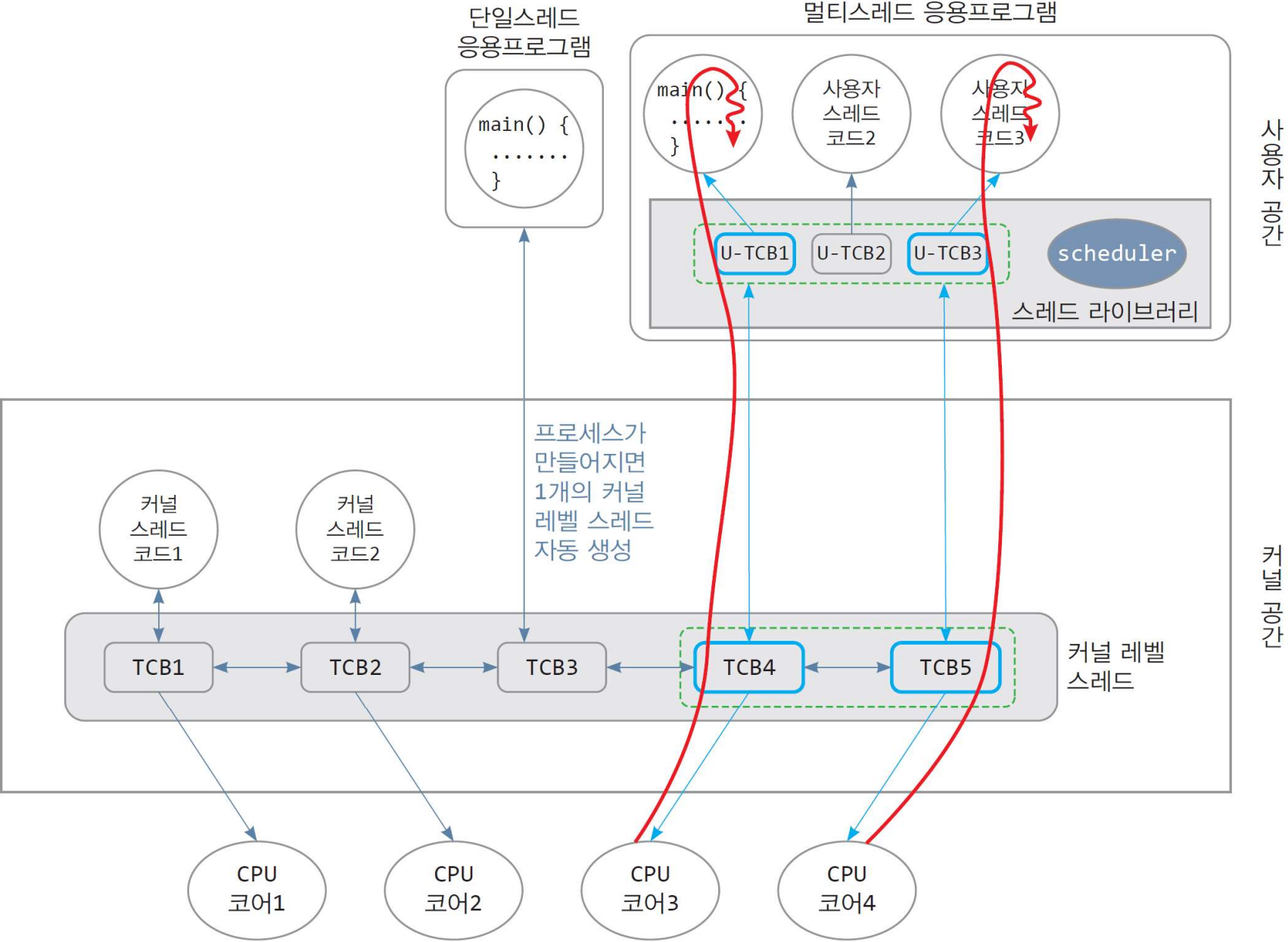


N:M 매핑

58

- N:M 매핑 개념
 - ▣ N개의 사용자 레벨 스레드를 M 개의 커널 레벨 스레드에 매핑
- 장단점
 - ▣ 장점
 - 1:1 매핑에 비해 커널 엔터티 개수가 작아 커널의 부담이 적음
 - ▣ 단점
 - 구현하기 복잡하여 현대의 운영체제에서는 거의 사용되지 않음

N:M 매핑 사례



6. 멀티스레딩에 관한 이슈

프로세스와 스레드 리뷰

61

- ▣ 프로세스는 스레드들의 공유 공간
 - 모든 스레드의 주소 공간이 프로세스 주소 공간 내에 형성되고 공유
- ▣ 프로세스는 운영체제가 응용프로그램을 **적재하는 단위**이고, 스레드는 **실행 단위**
- ▣ PCB에 저장된 정보는 **환경 컨텍스트**, TCB에 저장된 정보는 **실행 컨텍스트**
- ▣ 다른 프로세스에 속한 스레드로의 스위칭보다 동일한 프로세스에 속한 스레드 스위칭 속도가 빠름
- ▣ 프로세스에 속한 모든 스레드가 종료할 때 프로세스 종료

멀티스레딩으로 응용프로그램을 작성하는 장점

62

- ▣ 높은 실행 성능
 - 병렬 실행
- ▣ 사용자에게 대한 우수한 응답성
 - 한 스레드가 블록되어도 다른 스레드를 통해 사용자와의 입출력 가능
- ▣ 서버 프로그램의 우수한 응답성
 - 웹 서버나 파일 서버 등은 동시에 많은 사용자들의 접근
 - 이들을 병렬적으로 서비스하는데 우수
- ▣ 시스템 자원 사용의 효율성
 - 스레드는 프로세스에 비해 생성, 유지 시 메모리나 자원 적게 사용
- ▣ 응용프로그램 구조의 단순화
 - 작업 기준으로 응용프로그램을 여러 함수로 분할하고, 각 함수 별로 스레드를 만들어 동시 실행
 - 새로운 기능 추가 용이, 프로그램의 높은 확장성
- ▣ 작성이 쉽고 효율적인 통신

멀티스레딩에 있어 주의할 점

63

- 여러 개의 스레드 중 한 스레드가 `fork()`를 호출한 경우
 - ▣ 새로 생성된 프로세스는 `fork()`를 호출한 스레드로만 구성
 - ▣ 심각한 문제
- 한 스레드가 `exec()`를 호출한 경우
 - ▣ 현재 프로세스에 새 응용프로그램이 적재되어,
 - ▣ 현재 프로세스의 모든 스레드가 사라지는 문제
- 스레드 사이의 동기화 문제
 - ▣ 프로세스 내에 있는 공유 데이터를 다수 스레드가 액세스할 때, 공유 데이터 훼손 가능성 -> 동기화 기법으로 해결(6장)